

Article

Digital Twin for Designing Logic Gates in Minecraft Through Automated Circuit Verification and Real-Time Simulation

David Cruz García *, Isabel Alonso Correa, Sergio García González , Arturo Álvarez Sánchez 
and Gabriel Villarrubia González 

Expert Systems and Applications Laboratory, Faculty of Science, University of Salamanca,
37008 Salamanca, Spain; isabel.alonso@usal.es (I.A.C.); sergio.gg@usal.es (S.G.G.); arturoas@usal.es (A.Á.S.);
gvg@usal.es (G.V.G.)

* Correspondence: david.cruz.garciaa@usal.es

Abstract

This article presents a gamified digital twin in Minecraft designed to support practical exercises in digital logic in the Computer Engineering I course at the University of Salamanca. Implemented as a Spigot/Paper server plugin based on the Platform for Automatic coNstruction of orGanizations of intElligent Agents (PANGEA) multi-agent architecture, the system orchestrates four virtual organizations and employs a world cloning strategy (via Multiverse and WorldGuard) to ensure individual and isolated workspaces, while also enabling collaborative work. The central contribution is a multi-agent system with an integrated ‘black box’ verification engine that mitigates redstone asynchrony and latency through controlled signal injection and software clock synchronization, enabling real-time deterministic validation of both basic logic gates and more complex sequential circuits. Additionally, the ecosystem includes a specialized suite of logic scenarios and a web-based dashboard for real-time teacher monitoring. In a pilot study ($N = 30$), the system achieved an average task completion rate of 89.1%, and an adapted Unified Theory of Acceptance and Use of Technology (UTAUT) analysis indicated that technical stability is positively associated with student performance.

Keywords: digital twin; minecraft; STEM education; Java plugin development; logic gates

1. Introduction

Digital logic and logic circuit design are fundamental parts of contemporary computer architecture. From signal processing to the development of complex microarchitectures, the ability to adapt Boolean algebra abstractions into physical hardware implementations is a critical skill. In academia, mastery of this transition from symbolic formulas to logic gate topologies defines the foundation of digital computing. However, teaching and prototyping these systems face significant challenges. Traditional methods based on physical laboratories, which employ breadboards, integrated circuits, and manual wiring, while valuable for their tangibility, present critical problems of scalability, maintenance costs, and a high propensity for physical connection errors that distract the student from the fundamental logical problem [1].

On the other hand, there are educational software programs such as Logisim or Verilog that offer unquestionable technical precision. However, these environments are often single-user applications with static interfaces that do not encourage real-time collaboration or playful experimentation. There is a technical gap between formal theory and the practical



Academic Editor: George A.
Tsihrintzis

Received: 22 December 2025

Revised: 13 January 2026

Accepted: 19 January 2026

Published: 23 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

intuition needed to debug complex digital systems [2]. In this context, there is a need for platforms that combine the rigor of binary logic with simulation environments capable of supporting multiple users interacting synchronously in the same design space.

In a broader context, advanced simulation frameworks and digital twins have evolved significantly beyond their initial scope. Recent literature illustrates their versatility in diverse sectors, ranging from industrial IoT (Internet of Things integration using VR [3] and automotive hardware-in-the-loop systems [4], to robust multimodal data analysis frameworks [5]. Bringing these immersive and interactive principles into the educational domain is crucial for modernizing learning methodologies.

Over the last decade, the sandbox video game Minecraft has transcended its recreational purpose and is increasingly used in education. Its internal simulation engine, based on the material 'redstone', functions as a signal propagation system that emulates the behavior of digital circuits with binary states (on/off) [6,7]. The redstone system in Minecraft is fundamentally based on logic gates in the same way as in a digital electronic system, which means that it has the computational capacity to emulate any logic system, from a simple NAND gate to a functional 8-bit CPU with registers and memory [6]. However, the implementation of complex logic in the standard Minecraft environment presents several technical obstacles, including the absence of debugging tools, the difficulty of automatically verifying the correctness of circuits, and limitations in creating suitable environments (worlds) for their development [8].

This article describes the development and implementation of a specialized plugin for Minecraft servers, which aims to facilitate the technical analysis of logic circuits simulated in a virtual environment. Unlike previous pedagogical approaches based on Minecraft: Education Edition, the proposed solution employs a Spigot/Paper-compatible server architecture, enabling real-time event monitoring on any dedicated server.

The main technical contribution of this work lies in the development of a comprehensive functional validation engine integrated into the Minecraft environment. Unlike simple visual simulations, this system implements a state analysis algorithm that rigorously evaluates each output of the truth table of the designed circuit. The plugin subjects the constructed topology to a complete test cycle, systematically switching all possible input combinations to ensure that the logical response is identical to the theoretical model. This debugging capability extends to complex digital circuits, such as JK and RS flip-flops, where the engine verifies not only stable states, but also memory conditions and state transitions. The result is a virtual instrumentation tool that guarantees the logical correctness of the logic gates or digital circuits that are made in the Computer Engineering I course of the computational engineering degree at the University of Salamanca, allowing users to validate complex designs at the touch of a button.

The rest of the article is organized as follows: Section 2 reviews the background on gamified digital twins and the use of Minecraft in STEM (Science, Technology, Engineering, and Mathematics) education, and then defines the technical objectives of the study in Section 3. Section 4 details the methodology used, the design of the environment, and the assessment tools, leading into Section 5, which describes the proposed multi-agent architecture and the operation of the validation engine. Finally, Section 6 presents the analysis of the technical results and student perceptions, concluding with the conclusions and future directions in Section 7.

Contributions

Specifically, this article presents the following three main contributions to research:

1. An engineering method for deterministic verification implemented within a non-deterministic game engine, using controlled clock injection to validate Redstone circuits against formal truth tables.
2. An architecture based on MAS (multi-agent systems) capable of orchestrating autonomous agents to enable scalable, multi-user validation of educational circuits directly within the Minecraft environment.
3. A suite of scenarios and a monitoring platform were evaluated for Computer Engineering education and validated through a pilot study assessing both technical performance and student acceptance.

2. Background

In recent years, there has been a growing interest in integrating interactive digital technologies and virtual environments into STEM education, in order to improve interactivity, collaboration and connection with real learning contexts. Key areas underpinning this work include the use of virtual worlds such as Minecraft for the simulation of logic circuits and gates, the development of gamified educational environments that encourage computational exploration and thinking, and the implementation of accessible online platforms that promote participation and collaboration among students.

Digital twins have established themselves as essential tools in sectors such as Industry 4.0 for optimizing manufacturing, healthcare for simulating complex surgical procedures, and automotive for real-time vehicle monitoring [9]. However, an emerging and disruptive application is the use of Minecraft as a “serious gaming” platform for urban planning and social interaction. Thanks to its block-by-block construction gameplay, Minecraft allows users to recreate identical virtual versions of physical areas, such as the historic district of Dadaocheng in Taipei, facilitating co-creation and creative learning through play [10]. These environments allow citizens and experts to explore urban spaces through 3D avatars, analyzing the development process and collaborating on the design of future infrastructure before its actual implementation.

The proposed design is based on constructionism [11], promoting ‘learning by doing’ through the construction of logic circuits as public and tangible entities in a virtual environment. This technical architecture integrates experiential learning by transforming the design experience into knowledge through Kolb’s Experiential Learning Cycle [12], facilitated by immediate feedback and real-time simulation offered by the Digital Twin [13]. In this way, Minecraft functions as an immersive laboratory where the trial and error cycle allows students to experiment, reflect on automated verification, and actively apply new conceptual solutions to complex problems [14].

However, it is also crucial to acknowledge potential drawbacks identified in the literature, such as the risk of distraction due to ‘seductive details’ or the steep learning curve for students lacking gaming literacy [15].

The use of Minecraft as a geographical design tool stands out for its ability to encourage voluntary public participation, especially among young people, by simplifying complex technical concepts [16]. Specific applications such as Geocraft have used real LIDAR data to replicate the physical world and predict challenges in smart cities, while cities such as Zurich have transformed entire districts into Minecraft worlds to test more intuitive and playful participatory processes [10,16]. Recent research further supports the potential of gamified and game-based methods, such as Minecraft and similar interactive platforms, to empower communities and broaden participation in urban design processes, demonstrating that accessible, playful tools can foster co-creation and inclusivity in planning beyond traditional consultation formats [17].

Educational gamification and game-based learning have established themselves as effective strategies for active teaching and learning. Unlike game-based learning, gamification consists of applying elements and mechanics typical of video games, such as points, badges or leaderboards, in non-playful contexts, such as the classroom, with the aim of motivating and engaging students [18,19]. Game-based learning, on the other hand, uses a real video game, either digital or analog, as the main tool to teach skills and knowledge, favoring a more immersive learning experience. The design of an effective gamified educational environment seeks to generate a state of deep concentration in which students are fully involved in the activity. To achieve this, the design must adequately balance the playful and pedagogical components, considering the player's identity, their level of interaction and a gradual progression in the complexity of the tasks. For serious games, it is critical that the purpose, content, and narrative are aligned with the learning objectives [20].

Game-based learning is underpinned by theoretical approaches such as constructivism and experiential learning, in which students actively learn by solving problems in simulated environments. A key principle in the design of this type of experience is to achieve the immersion of the users participating in the activity. To do this, the environment must offer constant feedback, clear goals, and progressive, balanced challenges [21,22]. The design elements of an educational game are organized into three levels: the basic components (such as points or avatars), the mechanics (the rules of interaction between the player and the environment) and the dynamics (more abstract aspects such as emotion or the feeling of progress). Various theoretical frameworks emphasize that player identity, interactivity, and incremental complexity are essential factors in maintaining motivation. Recent studies identify six fundamental factors for the design of serious games: the purpose of the game, the validity of its content, the mechanics, the narrative or fiction that contextualizes it, the suitability of the target audience and the visual aesthetics [23]. Also, according to recent systematic analyses of serious games, the design of game scenarios as key components facilitates the integration of rules, mechanics, and immersive learning contexts [24]. Taken together, these contributions show that the success of educational games does not depend only on their technical or aesthetic elements, but on their ability to coherently articulate the playful experience with the pedagogical objectives, promoting meaningful and sustained learning over time.

The use of video games in educational environments has gained relevance as an innovative pedagogical tool, demonstrating its potential to improve student motivation, participation and academic performance [25]. Several studies have explored how video games can promote cognitive and socio-emotional skills, as well as facilitate the acquisition of skills in different fields of knowledge [26]. For example, research in technology education shows that video games applied to the learning of complex concepts allow for deeper understanding and active learning [27]. Similarly, it has been shown that gamification in university teaching environments enhances interactivity and student engagement with [28] content. Likewise, the use of video games in education has been shown to improve collaboration, problem-solving and decision-making, essential elements in modern academic training [29]. Finally, recent studies highlight that integrating video games requires a conscious pedagogical design, with strategies aligned with the curriculum and structured evaluation of the results to maximize their effectiveness [30].

The integration of environments such as Minecraft into university and school education has been the subject of increasing interest recently, reflecting its ability to foster creativity, collaboration, and critical thinking [31]. For example, it has explored how its open nature allows for forms of meaningful learning that transcend the traditional [32] classroom. More broadly, recent reviews have pointed out that game-based learning environments offer a useful framework for investigating interactivity, immersion, and linkage

to real learning contexts [33]. Likewise, studies focused on higher education and the computer science discipline have shown that gamified digital experimentation can transform traditional teaching-learning methods [34]. In an analysis focused on advanced educational environments, key factors, such as pedagogical alignment, adaptability to students and the effectiveness of feedback, have been identified for the success of these [35] interventions. Finally, recent research insists that the use of interactive digital platforms requires not only technical infrastructure, but also conscious and evidence-based didactic planning [36].

With this in mind, this section suggests that the adoption of Minecraft and similar environments in learning applications of concepts such as logic gates has clear potential, provided that proper design, curricular alignment, and structured assessment of outcomes are met [37].

In this line, the evolution of virtual environments for teaching logic and circuits has transitioned from gamification tools such as Minecraft: Education Edition, which allows logic operators to be modeled using “Redstone” in an immersive world, to sophisticated digital twin architectures that enhance spatial understanding and academic performance through six-dimensional models and real-time data [38–40]. However, current automated assessment and feedback systems often rely on rigid comparisons with desired solutions or dynamic unit tests, limiting their effectiveness in open-ended design projects or unstructured tasks [41,42]. Although proposals exist that utilize Formal Concept Analysis (FCA) and combinatorial testing to optimize topic coverage in electrical engineering assessments, many virtual laboratories still require complex installations or are constrained by the restrictions of standard educational versions compared to the flexibility of server architectures with plugins [38,40,43]. Our proposal differentiates itself technically by integrating a web-accessible digital twin framework with a verification system based on static and dynamic analysis, overcoming the dichotomy between structured code evaluation and the creative freedom of virtual environments [40,42]. This research identifies a critical gap: the lack of tools that combine the scalability of learning management systems with the technical precision of industrial digital twins for circuit error diagnosis [41–43]. Therefore, we explicitly claim as a novelty the creation of an ecosystem that unifies process-oriented immediate feedback with a decoupled architecture based on JSON (JavaScript Object Notation) data, facilitating a deeper knowledge transfer than traditional passive instruction methods [40,41]. Simply put, current virtual labs are like interactive textbooks that only accept one correct answer, whereas our approach functions as a digital workshop master that not only observes the final result but also understands and guides every circuit connection while the student builds it.

Finally, from a technical perspective, Minecraft’s Redstone mechanism has been documented in the literature as a Turing-complete system capable of simulating complex digital logic [44]. However, the standard game loop processes logic updates in discrete time units known as ‘game ticks.’ The server architecture (Spigot/Paper) imposes a fixed update loop where 20 game ticks equal one second in the real world [6]. Recent studies have validated this environment as a viable workspace for introductory digital electronics, provided that signal propagation delays are correctly managed to avoid race conditions [45].

3. Objectives of the Study

The main objective of this study is to develop and validate a Minecraft-based digital twin framework for the simulation and automatic validation of logic gates and circuits. The specific technical objectives are as follows:

- Multi-agent architecture: Design and implement a software architecture where users and validation modules act as autonomous agents, interacting within a

Spigot/Paper server environment for the construction and validation of logic gates made from Redstone.

- Deployment of scenarios: Configure a set of eight practical circuit challenges that progress in complexity, from basic logic gates to sequential (bistable) systems. These scenarios serve to validate the accuracy of the evaluation engine and confirm that the system correctly identifies user solutions at different levels of technical difficulty.
- Validation engine: Develop an evaluation algorithm that acts as a supervisory agent, validating the logic gates or circuits constructed through real-time analysis of truth tables.
- Implementation of a monitoring website: Develop a monitoring infrastructure (website and database) that acts as an agent, remotely recording the score of each student who completes the exercises.
- Validation of technological acceptance using UTAUT: Analyze operational feasibility and user experience (UX) using the UTAUT (Unified Theory of Acceptance and Use of Technology) model, correlating objective performance data (stored scores) with subjective perceptions of Perceived Ease of Use (PEOU) and technical usefulness.

4. Methodology

For this work, an exploratory case study methodology was adopted, focused on the technical validation and user acceptance (UAT) of the developed tool. The study was designed as a pilot test to evaluate the correct functioning of the architecture and the initial reception by the students before a large-scale implementation. The approach combines educational software engineering with a mixed-methods analysis (quantitative logs and qualitative perception).

4.1. General System Design

From a functional point of view, the educational environment is organized around a “central world” or hub (Figure 1), where all participants can interact, communicate and visualize the different portals of access to the exercises. Each portal corresponds to a level or logical challenge, based on a specific type of logic gate or combination of them.

This architectural decision to create isolated instances was not purely technical but pedagogically driven. By providing a private ‘sandbox,’ the environment fosters a safe space for constructivist experimentation, where students can iteratively build and fail without the social pressure of a shared classroom, while the automated feedback acts as a scaffolding mechanism to guide their reflective process.



Figure 1. View of the central world of the university server.

When the student passes through a portal, the system transports them to an individual world (a separate instance or independent world within the server). From a technical perspective, the use of optimized APIs (PaperMC) and restricted superflat worlds significantly optimizes resource consumption due to their simplified topology. Regarding storage, while maintaining multiple worlds might suggest high disk usage, the exercise environments are spatially restricted. Players are confined to specific delimited zones, preventing the procedural generation of new terrain chunks (16×16 block segments) outside the working area. Consequently, the accumulated disk space for these minimalist worlds is significantly lower than that of a single standard survival world where exploration continuously generates new terrain. This efficiency, combined with an event-driven validation algorithm that avoids continuous polling, ensures the system is theoretically scalable to high concurrency (e.g., $N \geq 30$ s) without performance degradation. Furthermore, this architecture is flexible: while the current configuration isolates each student to prevent interference during individual assessment, the system natively supports collaborative work, allowing multiple users to access and build within the same world instance if the pedagogical activity requires it. In this environment, the user must build, using redstone, the circuit that solves the proposed exercise. The exercises developed can be seen in Table 1.

Table 1. Description of each of the exercises performed. Exercises are detailed and correspond to the practical activities carried out in the simulation environment.

Ex.	Task Description
1	Complete an AND gate using redstone in Minecraft. Students must construct the door and verify its operation according to the truth table.
2	Implement the logical function $f_2(a, b, c) = a \cdot b + c$ with redstone, making sure that the output corresponds to the logical expression for all possible combinations of inputs.
3	Design a one-bit comparator that indicates whether the inputs are the same or different, using redstone components.
4	Build a 4-input (4:1) multiplexer in Minecraft, controlled by the selection signals, and check that the output correctly selects one of the four inputs.
5	Implement a 2-bit semi-adder, generating the addition and carry outputs, and verifying its correct operation with all input combinations.
6	Build a complete 2-bit adder, which correctly calculates addition and carryover, integrating the concepts of semi-adder and combinational logic gates.
7	Build an RS flip-flop that is provided with a clock to test its operation.
8	Build a bistable JK with asynchronous clear and preset inputs, which is also provided with a clock. (Figure 2)

In Figure 2, blocks of different colors can be seen, where the yellow and purple concrete serve solely as structural support. The red layout corresponds to the Redstone, representing the logical wiring of the circuit.

The plugin automatically validates the solution by detecting the final state of the circuit (for example, if the output matches the expected truth table). Once the exercise is completed, the player is returned to the hub through a portal and enabled to access the next level. This system ensures a sequential progression, preventing progress without having correctly solved the previous exercises.

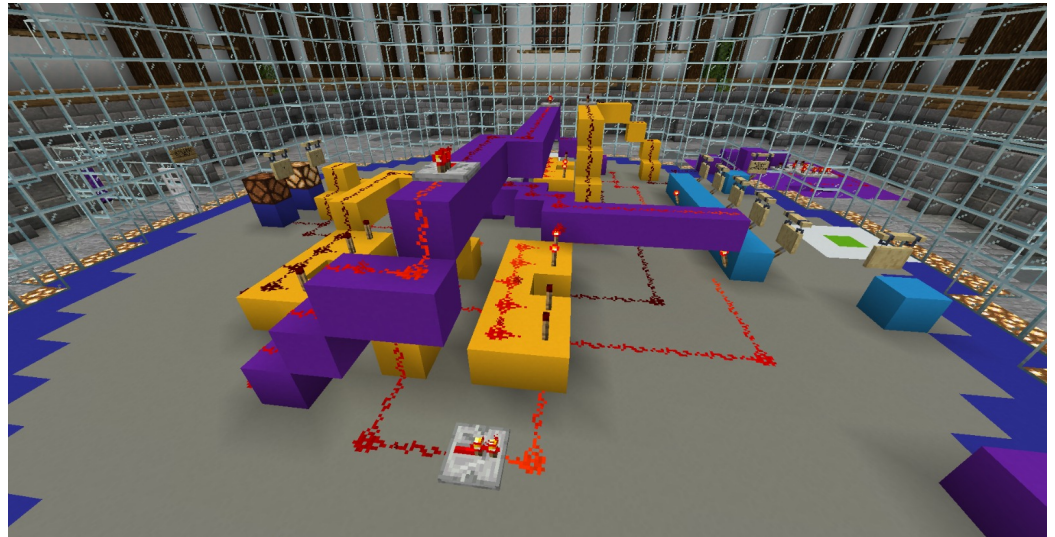


Figure 2. View of the final exercise solved (JK flip-flop).

4.2. Phases of Development

The methodological process was structured in the following four main phases:

- Analysis and definition of requirements: The curricular contents of the subject “Computer Engineering I” related to digital logic and logic gates were identified, taking as a reference the exercises proposed on page <http://avellano.fis.usal.es> [45].

In this phase, the functional requirements of the system (user control, progression, evaluation, and recording of scores) and the pedagogical requirements were defined, focusing on the clarity of the exercises, the balance between difficulty and motivation, and the correspondence with the learning objectives of the subject.

- Design of the environment: The server architecture was designed, defining the following two differentiated spaces:
 - Hub or main world: common area for meeting, orientation and selection of exercises through portals.
 - Individual exercise worlds: dynamically instantiated environments (thanks to the WorldEdit plugin) for each player, where practice takes place.

The visual and structural design of the worlds was aimed at maintaining a coherent and non-distracting atmosphere, facilitating the student’s concentration on the logical task.

- Technical development and implementation: The plugin was developed in Java, using the Maven dependency manager and the Spigot/Paper API. The main functionalities implemented were as follows:
 - Portal management and teleportation to individual worlds.
 - Sequential progression system by validating completed exercises.
 - Solution detection by reading redstone states and comparing them to expected output.
 - Implementation of a web-based platform to recover the scores obtained from the exercises.
 - Recording and storing scores in a local database.
 - Feedback interface through on-screen messages and information panels.
 - Development of a multiagent system to automate data collection, validation, and management tasks.

During this phase, principles of modularity and maintainability of the code were applied, in order to allow the future expansion of the repertoire of exercises or the incorporation of new evaluation mechanisms.

- Evaluation and pilot test: A technical validation was conducted with “Computer Engineering I” students, analyzing the following dimensions:
 - Infrastructure: Technical stability and available resources (Facilitating Conditions (FC)).
 - Profile: Prior experience with Minecraft and digital logic.
 - Performance: Objective scores.
 - Usability: Perceived Ease of Use (PEOU) and interface clarity.
 - Utility: Perceived pedagogical value for learning.
 - Adoption: Social Influence (SI) and future use intention.

Data were collected through direct observation, questionnaires and automatic server records, with the aim of analyzing the didactic effectiveness of the environment and its acceptance by users.

4.3. Parameter Calibration

The specific timing values used in this implementation were not arbitrary; they were established through a pre-pilot calibration phase based on Minecraft’s redstone mechanics and worst-case circuit analysis as follows:

- Physical baseline: A standard redstone torch introduces a delay of 2 ticks (0.1 s). The most complex circuit in the syllabus (Full Adder) has a logic depth of approximately 8 gates, implying a physical propagation time of 16 ticks.
- Safety margin: To accommodate server performance fluctuations without risking false negatives, a 25% safety margin was applied. Thus, the propagation delay (T_{prop}) was set to 20 ticks.
- Setup time: The setup time (T_{setup}) was configured to 40 ticks to account for the slower animation speed of mechanical components (pistons) compared to pure redstone dust, ensuring the circuit is physically stable before verification begins.

4.4. Participants

The study sample was composed of students of the Degree in Computer Engineering of the subject “Computer Engineering I” of the University of Salamanca. Specifically, a pilot test was conducted with 30 students selected from different laboratory groups (e.g., PA1, PB3) of the subject. The sample was selected for convenience based on their active participation in the laboratory sessions.

The participants were enrolled in the first year of the degree, aged between 18 and 22, and had basic knowledge of structured programming and fundamentals of digital logic, acquired in previous or parallel subjects of the curriculum.

The choice of these groups is based on their familiarity with the theoretical contents of the combinational and sequential logic, which allowed the evaluation of the impact of the virtual environment on the practical understanding and conceptual reinforcement of these contents. Participation was voluntary and not assessable, guaranteeing the confidentiality and anonymity of all the data obtained, in accordance with the institutional ethical guidelines for educational research.

4.5. Instruments

4.5.1. Objective Evaluation: Plugin Scoring System

The main instrument for measuring results was the plugin itself, implemented on the Minecraft educational server. This system acts as an automatic and objective evaluator of the student’s performance. The plugin takes care of the following:

- Performance log: The plugin records each participant’s total score and progression as they sequentially complete the eight hands-on digital logic exercises.

- **Solution validation:** Validation is performed automatically by detecting the final state of the redstone circuits built by the students and comparing them with the expected truth table or logical expression.

4.5.2. Subjective Assessment: POST-Intervention Questionnaire (UTAUT Model)

A POST-intervention questionnaire was administered to the students at the end of the practical activity to collect their opinion and experience about the tool and the general experience. The items were designed on a 5-point Likert scale (1 = Strongly disagree; 5 = Strongly agree).

The structure of the questionnaire follows the Unified Theory of Technology Acceptance and Use (UTAUT) [46] model, which allows evaluating not only the usability, but also the intention of future use and the impact of social factors on the adoption of the system.

Although participation is voluntary and the data is treated anonymously, guaranteeing confidentiality, the questionnaire included a field for the player's nickname. This identifier was essential to match the results of the survey with the actual score obtained by the student on the server, thus allowing correlation analyses between experience and performance.

The questionnaire was structured in five key constructs, derived from the following thematic sections:

- **Perceived Usefulness (PU):** (Section 2) Measures the degree to which the student believes the tool improved their understanding and the relationship between theory and practice.
- **PEOU:** (Section 3) Assesses the clarity of the interface, the installation of the software, and the intuitiveness of interaction in the virtual world.
- **SI:** (Section 4) Analyze the impact of peer recommendations, teacher support and the social climate on student motivation.
- **FC:** (Section 5) Measures the perception of the infrastructure, technical resources and guidance or support that were available to support the use of the tool.
- **Intention to Use (ITU):** (Section 6) Assesses the student's willingness to use the tool in the future and to recommend it to other peers.

To define these constructs and measure student perception, a set of 15 specific Likert scale items was designed (in addition to the objective and open data fields). The exact correspondence between each construct of the UTAUT theory and the items of the questionnaire is detailed in Table 2. This matrix not only ensures the validity of the instrument by replicating the underlying theoretical structure, but also facilitates the interpretation of the results in terms of the acceptance of the gamification environment.

Table 2. Full structure of the POST-intervention questionnaire.

Type	Field Name/Item	Description and Associated Construct (Likert)
A. Identification data and objective metrics		
Identifier	Nick in Minecraft	Unique identifier for pairing with server performance data.
Temporal	Timestamp	Date and time of completion of the survey.
Metric	Number of tasks/modules completed	Target performance metric.
Diagnostics	You had technical issues during the activity	Binary response (Yes/No) to assess infrastructure failures.

Table 2. *Cont.*

Type	Field Name/Item	Description and Associated Construct (Likert)
B. Previous experience and qualitative feedback		
Diagnosis	Previous experience with Minecraft	Assesses initial familiarity with the work environment (Yes/No).
Diagnosis	Previous experience with logic gates or digital logic	Assesses initial theoretical knowledge (Yes/No).
Qualitative (Open)	What concept of logic gates became clearer to you thanks to the activity?	Free response to get specific feedback on the pedagogical impact.
Qualitative (Open)	What would you improve about the plugin or the activity for future students?	Free response for getting suggestions for improvement.
C. Acceptance constructs (Likert Scale 1–5)		
PU	The plugin helped me better understand logic gates.	PU: Measures improvement in comprehension.
PU	The activity allowed me to relate theory and practice effectively.	PU: Measures the connection between theoretical concepts and their application.
PU	I learned concepts that I wouldn't have understood the same with the theoretical class alone.	PU: Measures the pedagogical added value of the tool.
PEOU	It was easy to install and configure the plugin.	PEOU: Measures initial technical usability.
PEOU	Interacting with the world of Minecraft was intuitive.	PEOU: Measures the usability of the virtual interface.
PEOU	The instructions provided were clear and sufficient.	PEOU: Measures the clarity and sufficiency of the supporting material.
SI	My colleagues would recommend using this tool.	SI: Measures peer group influence.
SI	The teacher supported or recommended the use of the plugin.	SI: Measures the influence of academic authority.
SI	Acceptance from my peers influenced my motivation to use the plugin.	SI: Measures internal social pressure.
FC	Had the necessary resources (computer, software) to use the plugin.	FC: Measures the availability of resources.
FC	I had the right guidance and support during the activity.	FC: Measures the perception of human support.
FC	Infrastructure (hardware/software) allowed the activity to run smoothly.	FC: Measures the quality of technical support.
ITU	I'd like to use this kind of tool in other subjects.	ITU: Measures intent to use in other contexts.
ITU	I would recommend this activity to other students.	ITU: Measures the future promotion of the activity.
ITU	I would use this plugin again in the future if I had the chance.	ITU: Measures the intention of future personal use.

4.6. Ethical Considerations and Data Privacy

This study was conducted as an educational innovation and technical pilot initiative within the framework of the “Computer Engineering I” course of the Computer Engineering Degree at the University of Salamanca. It was a multi-group pilot test and proof of concept (PoC) focused on software validation and usability, rather than an educational intervention. Participation was strictly voluntary, anonymous and non-assessable (it did not affect academic grades). All participants gave their informed consent for the use of their anonymized data for technical evaluation. The data was processed in accordance with data protection regulations (GDPR), ensuring that no sensitive personal information was stored.

Although the host institution did not require formal IRB approval due to the non-medical and non-invasive nature of the educational activity, the study strictly adhered

to standard ethical guidelines for research involving human participants. To ensure data privacy and mitigate the risk of social desirability bias, particularly given the academic setting, the following procedural measures were implemented:

- Informed consent and voluntariness: All participants were informed about the research purpose of the pilot study. Participation in the data analysis was strictly voluntary and non-assessable; students retained the right to opt out without any academic penalty.
- Anonymization: No personally identifiable information (PII) or institutional enrollment numbers were collected. Data correlation was based exclusively on arbitrary in-game nicknames, ensuring that the analyzed dataset remained completely dissociated from the students' real identities. The data was processed in accordance with data protection regulations (GDPR).
- Data retention: The collected data was stored on local servers restricted to the research team and will be retained only for as long as necessary for scientific validation.

5. Proposed Architecture

5.1. Platform Selection and Technological Choices

The selection of the technological stack and verification strategy was driven by a comparative analysis of available alternatives. The design decisions prioritized accessibility, scalability, and pedagogical flexibility over pure simulation performance as follows:

1. Server-side plugin (Spigot) vs. Client-side mods (Forge/Fabric): Traditional Minecraft modifications often require students to install specific loaders and modpacks on their local machines, creating significant technical barriers and version compatibility issues. By developing a server-side plugin using the Spigot API, we adopted a “zero-client-configuration” architecture. Students can connect with a standard, vanilla Minecraft client, shifting the complexity entirely to the server infrastructure.
2. Game engine vs. dedicated simulators (e.g., Logisim/Verilog): While dedicated tools offer nanosecond-precision, they typically lack native multi-user support. Minecraft provides an inherent collaborative spatial environment. Our architecture accepts the trade-off of lower simulation speed (tick-based) not only for engagement but as a deliberate pedagogical feature: unlike instantaneous software compilation, the sequential visualization of the verification cycle allows students to observe the circuit's behavior in real-time for each truth-table case, facilitating visual debugging and conceptual assimilation.
3. Black-box verification vs. structural analysis: A structural analysis approach would require checking the specific placement of every redstone dust and torch, forcing students to replicate a rigid reference design. Instead, we implemented a “black-box” verification engine that evaluates the input/output truth table. This design decision is pedagogically crucial: it validates the logic function rather than the topology, allowing students to explore diverse, valid physical implementations of the same gate (creative freedom) while still ensuring logical correctness.

5.2. PANGEA-Based Multi-Agent Architecture

The proposed architecture seeks to offer a robust, modular and scalable solution for the detection, validation and traceability of the design of digital circuits. The approach is focused on hardware simulation and not a theoretical approach, as the goal is to identify the practical application of Boolean logic and then determine the accuracy of the circuit through automated evaluation. Avoiding the use of closed virtual simulators, with high computational cost, is key to the scalability of the proposed system.

To achieve the objective of this research work, which is the automatic evaluation and practical reinforcement of digital logic, it is necessary to have an architecture with well-defined characteristics for the system to operate correctly.

For this purpose, an architecture based on virtual agents is modeled where each one works individually to achieve a common goal. One of the main needs that led to the design of the system using this architecture is the ability to design a system that performs the different tasks of world instantiation, simulation of inputs/outputs and performance classification in a scalable way.

The works of PANGAEA [47,48] have served as the basis for the proposed architecture. PANGAEA allows different agents to dynamically adapt to the computational needs of the system and services to scale on demand. When an agent joins the platform, they must communicate what services are available and can offer to other entities. Compared to other systems, the use of PANGAEA allows the creation of virtual organizations, facilitating the visual representation and hierarchical modeling of the system.

Using PANGAEA as a foundation ensures that agents can flexibly coordinate, share resources, and adapt to different workloads, which is key to processing large volumes of circuit simulations efficiently, ensuring scalability even with high concurrent user access.

Compared to existing systems such as SPADE, JADE, osBrain, or Python v3.0-based multi-agent libraries, the PANGAEA multi-agent platform offers native support for the creation of virtual organizations. These virtual organizations allow complex systems to be modeled and visually represented in a structured way. Furthermore, as an open-source platform, PANGAEA facilitates the reproducibility and future validation of results by other researchers.

JADE (Java Agent Development Framework) is one of the most mature and widely adopted platforms for the development of multi-agent systems (MAS) in both academic and industrial settings. It complies with the specifications of the Foundation for Intelligent Physical Agents (FIPA), in particular the FIPA Agent Communication Language (ACL), which standardizes communication between agents and supports the deployment of distributed and cooperative agent networks. However, JADE is primarily geared towards the development and management of individual agents, which limits the direct modeling of complex virtual organizations. The implementation of hierarchical structures or organization-level representations often requires additional modules and manual configuration.

The architecture is structured in four Virtual Organizations (VOs) as seen in Figure 3, each one specialized in a function within the general learning and evaluation process, working in coordination to cover the cycle from student interaction to teacher reporting.

From a pedagogical perspective, this modular architecture serves a dual purpose: it technically optimizes resources while educationally ensuring that each student operates in an isolated 'sandbox'. This prevents peer interference and fosters a safe environment for individual experimentation, free from the pressure of a shared workspace.

Below, each of these four organizations will be described in greater detail:

1. PANGAEA Organization (Management Platform)

This organization constitutes the operational and management core of the Multi-Agent System (MAS). Its main purpose is to ensure the proper coordination, supervision and administration of all other organizations, guaranteeing stability and traceability as follows:

- Manager Agent: Responsible for monitoring the overall health of the system (e.g., server load) and coordinating actions. If you detect an imbalance, you can redistribute tasks or restart services to ensure operational stability.

- Organization Agent: Manages virtual organizations, ensuring that each one operates within its boundaries and complies with security policies. It is crucial for load balancing and encryption of internal communications.
- Normative Agent: Ensures that all interactions between agents follow defined rules and protocols, verifying regulatory compliance (e.g., level unlock sequence).
- Information Agent: Manages the catalog of services that each agent offers (Directory), facilitating efficient collaboration between organizations.
- Database Agent: Has exclusive access to the internal database of the PANGAEA system to store and retrieve information related to the state of agents and infrastructure.

2. Organization Minecraft GBP (Game-Based Plugin)

This organization acts as the primary point of contact between the system and the user (the student). Through this VO, users interact with the Minecraft environment intuitively and receive feedback. Their primary responsibility is to manage the gameplay experience, lobby, and initial persistence of performance data as follows:

- Lobby Agent: Provides the initial world (in the Minecraft spawn) for students to interact with the system, defines areas to inspect (portals), and visualizes exercise images and preliminary results (ranking).
- Progression Agent: Responsible for defining the sequence and the unlocking logic. It receives the input parameters (player status) and generates the list of exercises with which it can interact.
- Worldclone Agent: In charge of programming and executing the instantiation of the exercise environments (cloned worlds) corresponding to the selected levels, ensuring that each student works in an isolated environment.
- Teleport Agent: Manages player movement. It is responsible for executing teleportation safely between the lobby and the exercise worlds, and vice versa.
- Feedback Agent: Responsible for managing the notifications and feedback that the system emits. Send success, error, and instruction messages directly to the learner interface.
- GameData Agent: Its main purpose is the persistence of game data. It stores the player's academic information (scores, nicknames).

3. Organization Evaluation

This organization operates as a hierarchical sub-organization of the Minecraft GBP Organization (2). He is responsible for executing all processes related to the simulation and analysis of digital logic. Their role is critical, since they implement validation algorithms to guarantee the objective correctness of the circuit built by the student as follows:

- CircuitEvaluator Agent: This is the orchestrator agent of the test, coordinating the SimulationAgent and RuleEngine Agent. Use the truth tables to analyze the built circuit. This agent processes the inputs and coordinates the simulation to generate a validation result that is passed to the GameData Agent.
- Simulation Agent: Responsible for simulating the change in state of inputs (levers) and analyzing the output (lamp) in the world of Minecraft, interacting with the plugin to translate the relative coordinates of Redstone blocks to binary states recognized by the system.
- RuleEngine Agent: This is responsible for comparing the actual output of the circuit with the expected logic. It performs the binary comparison between the output of the Simulation Agent and the correct truth table, issuing the final verdict.

4. Web Application Organization

Its main objective is to validate the effectiveness of the environment through the management of objective and subjective data, offering a management interface for teachers through the web application. This organization separates the game's persistence logic from the user management of the web platform itself, guaranteeing security and modularity as follows:

- **WebData Agent:** Acts as the web's database manager. It is responsible for storing and retrieving essential information for the faculty interface, such as credentials, user profiles, and course structure.
- **UsersManager Agent:** Responsible for managing users of the web system. Control registration, authentication, and access permissions for teachers and administrators.
- **CourseManager Agent:** Responsible for setting up classes or groups of students and assigning which students belong to each course. It makes it easy for teachers to filter and visualize the performance data of their classes.
- **Dashboard Agent:** Processes teacher queries and presents relevant statistics derived from circuit evaluation, generating the dashboard to understand the overall situation (e.g., mean task completion was 89.1%).
- **UTAUT Agent:** Responsible for managing the post-intervention perception survey. Its essential function is to match the qualitative answers of the questionnaire (based on the UTAUT model) with the objective performance data (score recorded by the plugin) obtained by the GameData Agent and the CircuitEvaluator Agent.

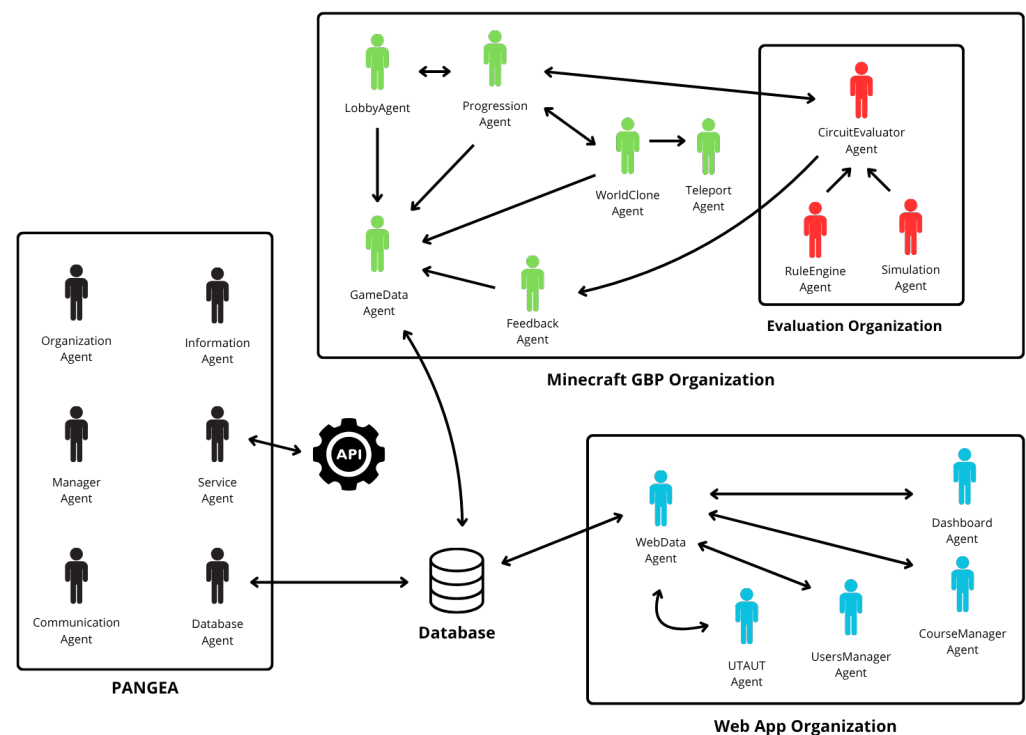


Figure 3. Multi-agent architecture of the developed environment (PANGAEA model).

The choice of a PANGAEA-based multi-agent system (MAS) not only responds to a need for modularity, but also to specific functional requirements. In particular, MAS allows for greater fault tolerance, as the Manager Agent monitors the status and can redistribute tasks or restart services, ensuring operational continuity. In addition, the architecture facilitates dynamic workload distribution, allowing agents from different organizations

(e.g., WorldClone Agent and Simulation Agent) to operate simultaneously on different nodes, processing simulations in parallel, which improves the scalability and efficiency of the system.

This system is the most suitable solution to achieve the robustness, scalability and modularity objectives required for automated monitoring of digital logic learning, overcoming the inherent limitations of monolithic architectures or conventional microservices.

5.3. General Operation of the Plugin

The core of the system is a Java-based plugin, which acts as a synchronization engine for the digital twin. This component translates redstone interactions into a logical state model, enabling the Minecraft environment to function as a testbed for the design and validation of logic gates.

5.3.1. Cloning Parallel Worlds

The architecture is designed to ensure that each student's activity is isolated and reproducible, avoiding interference (a key requirement for objective assessment). However, this isolation is flexible: the system permission logic allows for collaborative interaction, enabling students to enter a peer's world to provide assistance or work jointly within the same instance, thereby supporting cooperative learning strategies alongside individual evaluation. This function is orchestrated by the Progression Agent and executed by the WorldClone Agent (implemented in the WorldForPlayer clone method).

The cloning cycle, which ensures isolation from the environment, can be represented by the following simplified pseudocode in the WorldClone Agent:

```
FUNCTION cloneWorldForPlayer(player, base world)
  NEW_WORLD ← player.name + base world

  IF(MVWorldManager.exists(NEW_WORLD)) THEN
    WorldClone Agent.load(NEW_WORLD)
  ELSE
    MVWorldManager.cloneWorld(base world, NEW_WORLD)
    WorldGuard.cloneRegions(base world, NEW_WORLD)
  END IF

  Teleport Agent.move(player, NEW_WORLD.spawn)
END FUNCTION
```

1. Squad use and progression: In the main world hub, portals check the player's progression. If the progression is valid, the agent requests cloning.
2. Assisted cloning: The Multiverse-Core API is used to create an exact copy of the base world (e.g., exampleExercise1), naming the new world dynamically (Player-Name_exampleExercise1).
3. Region isolation: WorldGuard protection regions are cloned to prevent other players from accidentally modifying the circuits. In addition to the building permits, each template includes a chest with the Redstone materials needed to solve the circuit and an instruction book provided by the Feedback Agent detailing the objective of the exercise.
4. Instantiated teleport: Once cloned, the Teleport Agent teleports the student to the spawn point of the new practice environment.

This mechanism ensures that, even if several students simultaneously access the same exercise, each one operates in a private and controlled instance. Two different worlds are available, a smaller one for more compressed exercises (Figure 4) and a larger one for more extensive exercises (Figure 5) that act as insoles with their respective limited construction zones.



Figure 4. Short circuit exercise template.



Figure 5. Long circuit exercise template.

5.3.2. Circuit Testing

The validation of the students' designs is carried out automatically and sequentially, simulating the laboratory test to ensure the correction of the circuit in all its possible input combinations. This validation cycle is orchestrated by the CircuitEvaluator Agent and executed by the Simulation Agent and RuleEngine Agent.

The process starts when the student presses the check button in their exercise environment, executing the following sequence:

1. Starting the Test Loop: The CircuitEvaluator Agent notifies the Simulation Agent to start the systematic test loop. For a circuit with N inputs, 2^N combinations must be tried (e.g., $2^3 = 8$ for a full adder).
2. Input Simulation: The Simulation Agent uses scheduled tasks with fixed delays to simulate the changing state of physical inputs (levers) and read the state of outputs (lamps). The plugin code implements this using `Bukkit.getScheduler().runTaskLater`, ensuring a propagation time of the redstone signal before reading.
3. Sequential clock operation: For sequential circuits (such as RS and JK flip-flops), although the templates include an automatic clock as a reference, the simulation of the

test is performed via a dedicated manual clock lever. The Simulation Agent simulates the pulse of the clock (edge) by programmatically activating and deactivating this lever, ensuring that the circuit status is read at the precise moment of transition (up or down).

4. Logical validation: For each combination tested, the RuleEngine Agent (which contains the correct truth table) compares the binary output obtained from the student's circuit with the expected theoretical result.
5. Progressive Feedback: The Feedback Agent provides immediate feedback in the game chat, showing the current status of the entries and the percentage of the truth table that has been successfully verified.

This process is stopped and the student is informed if a single input combination results in an incorrect output, indicating that the circuit is valid or invalid. If all the combinations are correct, the success is recorded and the door to return to the main world is unlocked and the next level is allowed to be accessed.

Sample Check: Exercise 1 (AND Gate)

The validation process for the AND Gate ($S = A \cdot B$) illustrates the simulation cycle for a combinational circuit. The RuleEngine Agent applies the following logic following the truth table in Table 3.

1. Simulation of row 4 ($A = 1, B = 1$): The Simulation Agent activates lever A and lever B.
2. Read and compare: After propagation, the RuleEngine Agent reads the output S . If $S = 1$, the validation is [OK]. In Figure 6 you can see the check for the entries $A = 1, B = 0$.
3. In-game feedback: The Feedback Agent shows the player a message similar to: " $\rightarrow A = 1$ and $B = 1 \ S = 1$ [OK] [100%]", indicating that the simulation is correct for that combination, or incorrect if the circuit is poorly constructed and the test fails.

This reporting mechanism is the direct output of the validation engine detailed in Section 6.1. This process is detailed by service calls and collaboration between organizations in the sequence diagram (Figure 7) where communication between agents in the circuit validation process is shown.



Figure 6. Example check for input $A = 0, B = 1$.

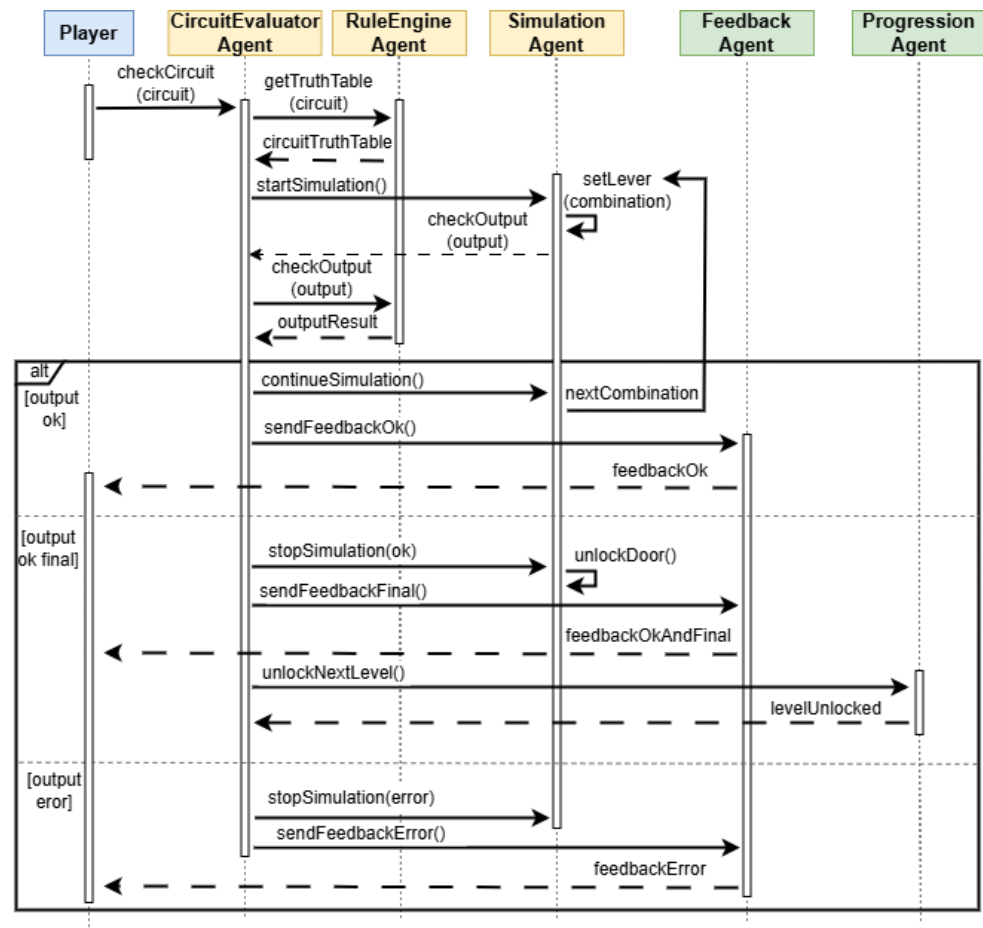


Figure 7. Sequence diagram of the automatic evaluation of the AND gate circuit.

Table 3. Exercise 1 truth table (AND Gate).

Input A	Input B	Output S
0	0	0
0	1	0
1	0	0
1	1	1

5.4. Web Platform Design and Functionality

Figure 8 shows the main interface of the web-based platform used to monitor the performance of students in the laboratory sessions of the subject Computer Engineering I, taught at the University of Salamanca. This tool has been designed with the aim of providing teachers with a clear and up-to-date view of their students' progress in the different activities proposed during the course. The interface, which is dynamic and interactive, allows you to consult in real time the individual and group evolution of the work carried out by the students in the assigned exercises.

At the top of the panel, a general summary is shown with the different groups of the course (PA1, PB3, PA2, etc.), indicating for each of them the number of students enrolled, the teacher in charge and the percentage of progress achieved. In this way, the teacher can quickly identify which groups show more significant progress and which require more attention or support.

Below this summary is a section of individual follow-up of the students, where the nicknames, group of belonging and number of completed exercises are listed compared to the total assigned. This information is continuously updated, which allows teachers to

monitor the performance of each student in detail and detect possible cases of delay or lack of participation.

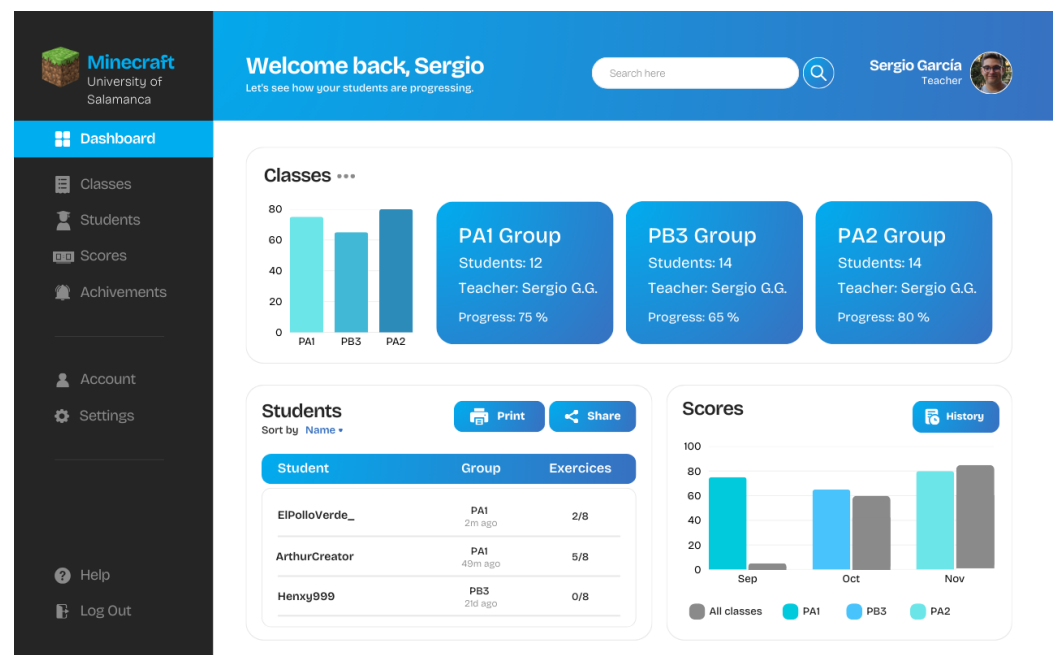


Figure 8. View of the web administration interface of the exercises.

The interface also incorporates a results analysis module, which graphically represents the evolution of the scores obtained by the different groups over time. These visualizations allow progress to be compared between laboratory groups, even those that are not directly associated with the teacher who consults the data, promoting a global and collaborative vision of academic performance in the subject.

Finally, the platform integrates additional functionalities such as printing and the exchange in results, as well as the possibility of filtering information according to specific criteria (name, group or period). Overall, this tool contributes to more efficient teacher management, supporting decision-making based on objective data and in real time.

5.5. Reproducibility and Technical Specifications

To ensure the replicability of the experiments and the stability of the results, the specific technical environment used is detailed below. No client-side mods are required for students, ensuring low barriers to entry as follows:

- Server software: Spigot MC 1.21.1 running on OpenJDK 21 (Headless).
- Hardware infrastructure: The pilot was hosted on a VPS with an Intel Xeon Skylake processor (2 vCores), 4 GB RAM, and SSD storage. Memory allocation for the JVM was constrained to 3 GB (-Xmx3G) to simulate a resource-limited environment.
- Client requirements: Minecraft Java Edition 1.21.1 (Vanilla).
- Configuration: The verification plugin was deployed with the calibrated parameters defined previously ($T_{prop} = 20$ ticks, $T_{setup} = 40$ ticks).

6. Results

The results presented in this section address two complementary dimensions: the technical robustness of the system and its pedagogical effectiveness. First, the technical validation of the automated assessment engine is detailed, focusing on the algorithms used to handle signal synchronization and state validation within the game physics. Subsequently, the empirical analysis carried out with a sample of $N \geq 30$ students of the Computer

Engineering Degree at the University of Salamanca is presented, structured into target performance, instrument validation (reliability), and perception analysis.

6.1. Technical Evaluation of the Developed Plugin

Before analyzing the student performance data, a technical validation of the developed plugin was conducted to ensure the reliability of the automated assessment engine. This phase was critical to distinguish between students' logical errors and potential false negatives caused by the system.

6.1.1. Validation Algorithm

Before analyzing student performance data, it is essential to describe the technical validation of the automated assessment engine developed. The core of the system interacts directly to manipulate and read the state of Redstone blocks in real time. The validation logic is based on a 'Black Box Testing' approach: the system injects the inputs defined in an internal truth table and reads the resulting outputs, independently of the architectural structure of the circuit built by the student. This deterministic approach ensures that any valid implementation of the logic function is accepted, regardless of whether the student used basic gates (AND/OR) or complex universal designs (NAND/NOR), thus encouraging a variety of solutions.

One of the most critical technical challenges addressed was synchronization with Minecraft's physics engine. Redstone signals are not instantaneous; they have an inherent propagation latency (measured in Game Ticks, where 1 tick is approximately 0.05 s). If the algorithm validated the output at the same instant as the input injection ($t = 0$), it would read the previous residual state, generating false negatives. To resolve this, a sampling logic based on task queues (Task Scheduling) was implemented. The system introduces a predetermined fixed delay between the signal injection and the output reading. This interval was set as a conservative standard value (sufficiently wide to exceed the maximum propagation time of the proposed exercises), thus ensuring signal stabilization without the need to analyze the specific depth of each circuit. Figure 9 illustrates this operational flow, detailing the sequence of stimulus injection and the specific timing delays applied to strictly validate both combinational and sequential logic.

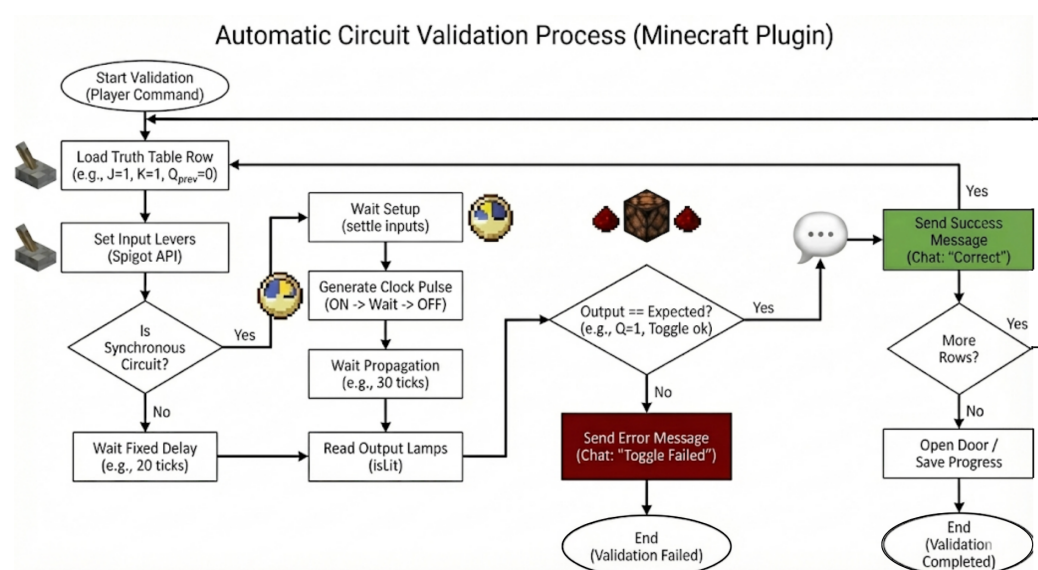


Figure 9. Flowchart of the automatic validation engine illustrating the stimulus injection process and the differentiated timing logic for combinational and synchronous sequential circuits.

Functionally, this validation loop acts as an automated scaffolding mechanism. Unlike manual grading, the algorithm provides immediate feedback, allowing students to engage in a continuous cycle of trial and error (re-enacting Kolb's active experimentation phase) without waiting for instructor intervention.

The algorithm architecture dynamically distinguishes between two types of circuits in order to apply the correct inputs and checks. For combinational circuits (e.g., logic gates, multiplexers), the output depends exclusively on the current inputs, so the system simply sets the states, waits for the standard propagation time, and validates. However, for synchronous sequential circuits (e.g., JK flip-flops, counters), where the state change depends on a clock edge, the plugin simulates a function generator.

It should be noted that, during the construction phase, the environment provides the student with an automatic clock (oscillator) connected to the circuit, allowing them to visually verify the dynamic operation of their design without having to generate edges manually. However, when the validation process begins, the algorithm forces this automatic clock to stop to avoid interference. In this exclusive control phase, the system not only sets the control inputs (J, K, R, S), but also synthesizes an artificial clock pulse by manipulating a dedicated lever: it generates a rising edge, maintains the high state for a fixed pulse width, and generates the falling edge, then waits for the circuit memory to update before comparing the result with the expected state table.

Figure 10 illustrates the interface for the control inputs (levers), while Figure 11 shows the output indicators and Figure 12 shows the automatic oscillator (clock) available during the construction phase.

The general pseudocode governing the check loop, which unifies the logic for both types of systems and manages student communication to facilitate debugging, is detailed in Algorithm 1.

By isolating this behavior, the plugin becomes a real-time visual debugger (as shown in Figure 13), an essential tool for students to understand sequential dynamics beyond simple static truth tables. Thanks to the granularity of the algorithm, the student receives feedback on the exact state that causes the failure, identifying, for example, a specific error in the 'Toggle' condition (see Figure 14). This state, critical in counter design, occurs in a JK flip-flop when both inputs are active ($J = 1, K = 1$), forcing the output to invert relative to its previous value with each clock pulse ($Q_{\text{next}} = \overline{Q_{\text{prev}}}$).

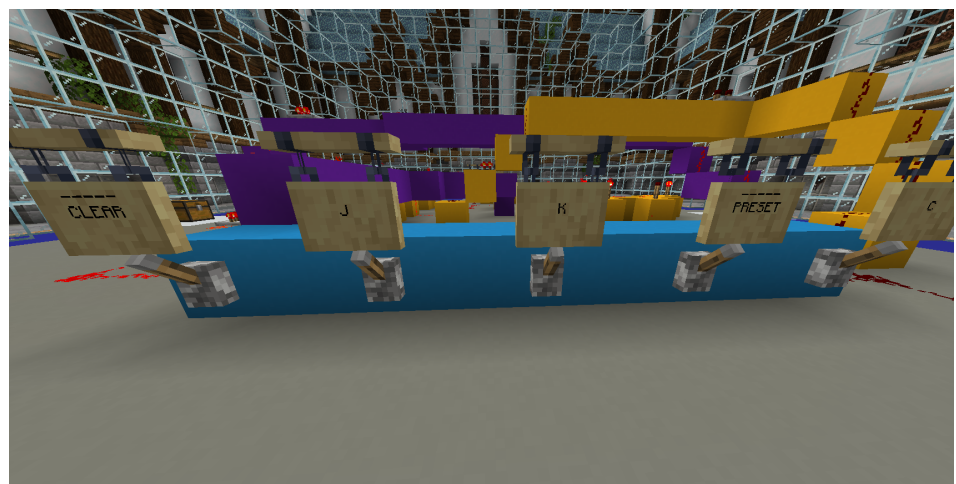


Figure 10. Input interface for the JK Flip-Flop exercise. The layout includes the synchronous inputs (J, K), the asynchronous controls (Preset, Clear), and the manual clock lever managed by the validation algorithm.



Figure 11. Output interface for the JK Flip-Flop exercise.



Figure 12. Automatic redstone oscillator (clock) provided to the student for dynamic testing during construction.

```

Iniciando prueba...
-> [10%/] Probando Clear Asíncrono (PR=1, CL=0)...
-> Q=0, Qn=1 [OK]
-> [20%/] Probando Preset Asíncrono (PR=0, CL=1)...
-> Q=1, Qn=0 [OK]
-> [30%/] Probando Inválido Asíncrono (PR=0, CL=0)...
-> Q=1, Qn=1 [OK]
-> [40%/] Estableciendo estado Q=0 para pruebas síncronas...
-> [50%/] Probando Memoria (J=0, K=0) desde Q=0...
-> Q=0, Qn=1 [OK]
-> [60%/] Probando Set (J=1, K=0)...
-> Q=1, Qn=0 [OK]
-> [70%/] Probando Reset (J=0, K=1)...
-> Q=0, Qn=1 [OK]
-> [80%/] Probando Complemento (J=1, K=1) desde Q=0...
-> Q=1, Qn=0 [OK]
-> [90%/] Probando Complemento (J=1, K=1) desde Q=1...
-> Q=0, Qn=1 [OK]
-> [100%/] ¡Comprobación completada!
Ejercicio Correcto [Puerta Abierta]
  
```

Figure 13. In-game chat displaying successful validation output.

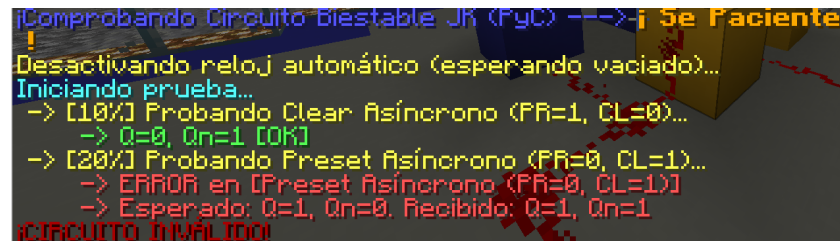


Figure 14. In-game chat displaying error validation output.

Algorithm 1. Unified circuit validation logic

Require: *TruthTable, CircuitType* (Combinational or Synchronous)

Ensure: Boolean *Pass*, *FeedbackMsg*

```

1: Initialize: Pass ← true
2: if CircuitType is Synchronous then
3:   DISABLEAUTOCLOCK                                ▷ Stop the visual oscillator
4:   WAIT(40 ticks)                                    ▷ Wait for signal stabilization
5: end if
6: for each TestRow in TruthTable do
7:   SETINPUTLEVERS(TestRow.Inputs)
8:   SENDPLAYERMSG("Testing State: " + TestRow.Description)
9:   if CircuitType is Combinational then
10:    WAIT(20 ticks)                                ▷ Standard propagation delay
11:   else if CircuitType is Synchronous then
12:    WAIT(Tsetup)                                    ▷ Allow J/K/Inputs to settle
13:    ClockLever ← ON                                ▷ Generate Rising Edge
14:    WAIT(Tpulse)                                    ▷ Pulse width (10 ticks)
15:    ClockLever ← OFF                                ▷ Generate Falling Edge
16:    WAIT(Tpropagation)                                ▷ Wait for memory update
17:   end if
18:   ActualOutput ← READREDSTONELAMPS
19:   if ActualOutput ≠ TestRow.ExpectedOutput then
20:     SENDERRORMSG("Mismatch at: " + TestRow.Description)
21:     SENDERRORMSG("Expected: " + TestRow.ExpectedOutput + " | Got: " +
ActualOutput)
22:     Pass ← false
23:     Break Loop
24:   end if
25: end for
26: if Pass is true then
27:   UNLOCKNEXTLEVEL
28:   SAVEPROGRESS
29:   SENDSUCCESSMSG("Circuit Verified Successfully")
30: end if

```

6.1.2. Sampling Logic and Delay Analysis

The implementation of the automatic validation system on the game engine has imposed synchronization challenges that do not exist in traditional circuit simulators. While tools such as SPICE operate under continuous time or optimized discrete event models, the Minecraft engine introduces fixed update physics (game loop) that acts as a hard constraint for validation. The execution environment is governed by discrete time units, where the game update cycle (T_{game}) operates at 20 Hz (20 updates per second), setting the system update rate at $T_{game} = 0.05$ s. However, *Redstone* logic is updated every two game cycles, defining the *Redstone Tick* (T_{rs}) as the atomic unit of signal propagation:

$$T_{rs} = 2 \cdot T_{game} = 0.1 \text{ s} \quad (1)$$

In terms of graph theory, the student circuit behaves like a network where each active component (torch, repeater) introduces a temporary weight of $1T_{rs}$. Consequently, the stabilization time (T_{stab}) required for an input signal to propagate to the output is a direct function of the logical depth (L) of the circuit's critical path:

$$T_{stab}(L) \geq \sum_{i=1}^L \delta_i \cdot T_{rs} \quad (2)$$

This latency generates a fundamental problem called a 'Race Condition' between the execution of the plugin code and the updating of blocks in the virtual world. If the algorithm injected the inputs (I_{t0}) and read the outputs in the same execution cycle, the system would obtain the residual state of the circuit prior to the change, i.e., $O_{read} \neq f(I_{t0})$.

To resolve this asynchrony in combinational circuits, a sampling delay (Δt_{sample}) was implemented, calculated to cover the reasonable 'worst case' in an educational environment. A conservative value of 20 ticks (1 s) was selected for the wait time before reading. This choice ($\Delta t_{sample} = 20T_{rs}$) is mathematically justified to ensure stability in circuits with a logic depth of up to 20 gates in series, which exceeds the expected complexity in basic academic exercises (where the maximum expected length is $L_{max} \approx 6$) by a wide margin of safety, without excessively penalizing the user experience with long waiting times.

The problem is exacerbated in the validation of sequential logic (bistable devices, counters), where dependence on time and clock edges makes the use of a fixed delay insufficient. The use of oscillators built by students introduces critical failure vectors, such as jitter (frequency instability) and violation of setup times. In Minecraft physics, if the clock signal reaches the bistable before the data signal due to differences in wiring length, an erroneous state occurs. To mitigate this, the developed algorithm overrides the automatic clock and takes full control of the cycle using a software state machine that enforces a deterministic sequence:

$$T_{ciclo} = T_{setup} + T_{pulse_width} + T_{prop} \quad (3)$$

In this sequence, the times have been specifically calibrated to neutralize the peculiarities of the game engine. A T_{setup} of 40 game ticks (2 s) was established, a deliberately high value to ensure that, even in circuits with inefficient wiring and very long paths, the control signals J and K are completely stable at the Flip-Flop inputs before generating the edge. Subsequently, a clock pulse with a duration of $T_{pulse_width} = 10$ ticks is generated. This pulse width is critical: it must be long enough to be registered by the repeaters (which filter pulses shorter than 1 tick) and avoid glitches, but controlled to define a clear edge. This converts an asynchronous and uncertain physical process into a verifiable logical evaluation, where the final output is validated only after ensuring complete propagation:

$$Q_{valid} = ReadState(t_{start} + T_{setup} + T_{pulse_width} + T_{prop}) \quad (4)$$

6.1.3. Sequential Circuit Management (Clock and Ticks)

In designing sequential logic exercises, it was observed that building stable oscillators represents a significant technical barrier for students, diverting attention from the main learning objectives (the behavior of bistable devices and counters). Poorly designed red-stone clocks often suffer from irregular frequencies or asymmetric periods. To isolate the evaluation of sequential logic from the physical implementation of the oscillator, the environment provides students with a pre-built automatic clock during the design phase. This allows the student to visually verify the dynamic behavior of their circuit (e.g., the flashing of a flip-flop) without having to generate pulses manually.

However, for the automatic evaluation phase, the presence of this free-running clock introduces a critical asynchrony problem. If an attempt were made to validate the circuit while the automatic clock is active, it would be impossible to predict which phase of the cycle (high or low level) the system is in at the exact moment the tests are injected, which would invalidate any deterministic reading of the output.

To resolve this conflict, a ‘software-controlled clock injection’ strategy has been implemented, executed directly by the Simulation Agent. Upon receiving the validation command, the agent activates an interrupt routine (DisableAutoClock) that cuts power to the student’s visual oscillator, stopping the circuit in a known state. From this point on, the Simulation Agent assumes complete control of the clock signal (CLK) by directly manipulating a dedicated control lever, turning the system under test into a synchronous state machine managed by the server scheduler.

The generation of clock edges is not instantaneous; the agent orchestrates a strict timed sequence, calibrated in Game Ticks, to ensure signal integrity within the Minecraft physics engine. This timing sequence is illustrated in Figure 15.

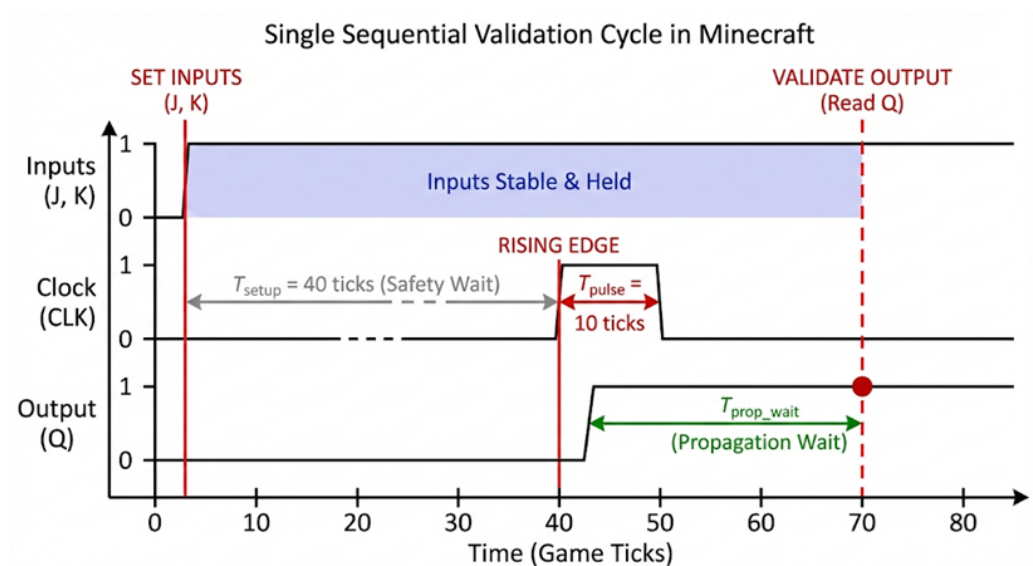


Figure 15. Timing diagram of the sequential validation cycle, illustrating controlled clock generation and the sampling instant.

The cyclical process executed by the agent for each test state consists of the following phases:

- **Setup time (T_{setup}):** After setting the control inputs, the agent introduces a wait of 40 Game Ticks (2 s). This interval, analogous to the safety margins defined in the previous section, ensures absolute stability of the signals at the circuit inputs before generating the clock edge, similar to clockless sequential circuits.
- **Edge and pulse generation (T_{pulse}):** The agent forces the clock lever to the ON state, generating a rising edge. This signal is deliberately kept active for 10 Game Ticks. This pulse width is a critical design decision: it is long enough to ensure the capacitive charging of the components and prevent the game repeaters from filtering the signal (which would occur with pulses < 1 tick), but short enough to define a clear firing cycle.
- **Post-edge validation:** Finally, the agent forces the clock to OFF (falling edge) and waits for the necessary propagation time (T_{prop_wait}) before reading the final state of the Q output.

This process executed by the Simulation Agent allows the validation logic to be perfectly synchronized with the game physics, ensuring that the truth table check occurs exactly after the edge has triggered the state transition, regardless of server latency.

6.1.4. Technical Performance Metrics

To validate the operational viability of the system, empirical metrics were recorded during the pilot sessions regarding validation times and server stability. The following apply:

- **Server configuration:** The pilot was hosted on a Virtual Private Server (VPS) equipped with an Intel Xeon Skylake processor (2 vCores), 4 GB of RAM, and SSD storage, running Spigot 1.20.
- **Validation times:** Table 4 presents the breakdown of times by type of exercise. The ‘Theoretical Time’ is calculated based on the configured delays ($T_{cycle} = 70$ ticks/case), while the ‘Observed Time’ reflects the actual duration including internal agent processing and/or server delays.
- **Server load (TPS):** During the laboratory sessions ($N = 30$ concurrent users), the server’s Ticks Per Second (TPS) were monitored. The system maintained a stable average of 19.95 TPS, with momentary drops to 18.5 TPS only during simultaneous world creation events. No significant latency was observed during the execution of the validation agents, confirming that Redstone’s logical evaluation has a negligible impact on server performance compared to chunk generation for a low number of simultaneous players.

Table 4. Average validation times and complexity by type of exercise.

Type of Exercise	Inputs (N)	Cases (2^N)	Theoretical Time	Observed Time (Average)
Basic Gates (AND/OR)	2	4	14.0 s	14.2 s
Combinational (Mux 4:1)	3 *	8	28.0 s	28.5 s
Sequential (Flip-Flops)	3	8	28.0 s	29.1 s
Counters/Adder	4	16	56.0 s	57.3 s

* Includes selection signals as inputs.

6.2. Performance and Activity Context (Objective and Descriptive Metrics)

This section validates the effectiveness of the environment in achieving learning objectives and establishes the context of the study group.

6.2.1. Context and Previous Experience

The analysis of the contextual variables (previous experience and resources) revealed the following study group with a high potential for the activity, although with a heterogeneous profile suitable for validating the learning curve:

- **Previous experience with logic:** Through the question “Previous experience with logic gates or digital logic”, 90% of the participants (27 out of 30) reported having previous experience with logic gates or digital logic. This data confirms that for the vast majority, the environment was used as a tool for practical reinforcement. However, the presence of a 10% of students without prior experience allowed validating the effectiveness of the plugin as an introductory tool as well.
- **Experience with Minecraft:** Through the question “Previous experience with Minecraft”, 70% of the students (21 out of 30) had previous experience with Minecraft. This indicates that while a majority was familiar with the mechanics, a significant 30% of the group faced the interface for the first time, which adds value to the high usability results obtained later.

Regarding the support of resources and infrastructure, it was evaluated whether the technical and logistical conditions adequately support the activity. Within the framework of the UTAUT model, this dimension corresponds to the FC, a critical factor that determines whether the user feels that the necessary infrastructure is available to use the system. The assessment was based on a 5-point Likert scale (where 1 = Strongly Disagree and 5 = Strongly Agree).

The joint analysis of the items that make up the FC (Section 5 of the survey) revealed a high level of satisfaction as seen in Table 5.

Table 5. Results of the facilitating conditions assessment ($N = 30$).

Key Item	Likert Average (X/5)	Detail and Implication
Resources and software	4.60	The vast majority of students indicated that they had the necessary resources (computer, software) available.
Technical infrastructure	4.63	This item measures whether “The infrastructure (hardware/software) allowed the activity to run smoothly”. The high average validates the robustness of the server and the plugin, with only isolated exceptions.

A mean of 4.62 on a scale of 5 indicates a level of strong agreement on the part of the participants. This result is crucial, as it shows that:

1. Accessibility: Hardware and software requirements were accessible to students.
2. Technical stability: The plugin development and server architecture worked reliably during the uptime, which is a significant achievement for a custom system.

In summary, the initial context of the group, characterized by a high theoretical base but mixed familiarity with the game, was complemented by a stable and well-supported software environment (high FC), creating the optimal conditions to evaluate the pedagogical impact of the plugin.

6.2.2. Performance and Productivity (Objective Metrics)

The target performance was evaluated by the final score recorded by the plugin (maximum 8 points), analyzing the completion rate of the proposed logic challenges. The analysis revealed that the group’s performance was high and consistent across the sample.

Performance was assessed through the number of tasks/modules completed (plugin score), with a maximum of 8 exercises. Descriptive performance statistics can be found in Table 6.

Table 6. Objective performance metrics in activity ($N = 30$).

Performance Metric	Value	Interpretation
Average of completed tasks	7.13/8.00	The average student successfully completed 89.1% of the proposed exercises, showing a high degree of efficacy.
Standard deviation (σ)	0.80	The low dispersion indicates a highly consistent performance across the entire group, with few outliers.
Maximum achievement rate	36.7% (Score = 8)	More than a third of the sample (11 students) managed to complete all 8 logic exercises, including those of advanced complexity.

It should be noted that Table 6 focuses exclusively on the performance within the Minecraft environment. A direct quantitative comparison with traditional methods (e.g., Verilog) was not conducted in this study because, in the standard curriculum of the subject, these specific exercises are optional self-study tasks and are not systematically tracked. For this academic year, the pilot prioritized the validation of voluntary adoption in the Digital Twin, and thus the collection of control data from Verilog users was not enforced.

The high average score confirms the effectiveness of the Minecraft environment in facilitating the practical translation of Boolean logic into circuit building.

Operational stability was measured by the incidence of technical interruptions (question “Did you have technical problems during the activity”). The Table 7 shows the stability metrics.

Table 7. Technical stability metrics ($N = 30$).

Stability Metric	Value	Detail
Incidence of technical problems	0% reported critical issues	The absence of reported blocking problems confirms the operational robustness of the plugin and the infrastructure, allowing the learner to concentrate on the logical task.

6.3. Instrument Validation (Reliability)

To ensure the validity and methodological rigor of the perception analysis, the internal consistency of the item groupings was evaluated by calculating Cronbach’s alpha (α) [49]. Cronbach’s Alpha is a coefficient that measures the internal consistency of a scale or construct. In essence, it assesses whether a set of items (questions in the same section, e.g., “PU”) consistently measure the same concept. The formula for calculating this metric is as follows:

$$\alpha = \left(\frac{k}{k-1} \right) \left(1 - \frac{\sum_{i=1}^k \sigma_i^2}{\sigma_t^2} \right) \quad (5)$$

where the following apply:

- k : This is the total number of items (questions) that make up the construct or scale (in this case, $k = 3$ for each section).
- $\sum_{i=1}^k \sigma_i^2$: It is the sum of the variances of the scores of each individual item (the variance of question 1 + the variance of question 2 + ...).
- σ_t^2 : This is the variance of the total observed score (the variance of the sum of the scores of all items).

The results of internal consistency, based on the items in Sections 2–6 of the survey and calculated using Cronbach’s Alpha (α) for the sample of $N = 30$, are shown in Table 8. As can be seen, all perception scales reach high levels of reliability, benefiting from the larger sample size.

Table 8. Results of Cronbach’s Alpha (α) by construct ($N = 30$).

Construct (Section)	Question Base	Cronbach’s Alpha (α)	Reliability Conclusion
I. PU	Section 2	0.91	Excellent reliability.
II. PEOU	Section 3	0.82	High reliability.
III. SI	Section 4	0.80	High reliability.
IV. FC	Section 5	0.83	High reliability.
V. ITU	Section 6	0.93	Excellent reliability.

The obtaining of Chronbach's Alpha (α) values consistently higher than the acceptable threshold of 0.70 for all constructs validates the reliability of the instrument and fully justifies the use of composite means for perception analysis.

6.4. Perception and Acceptance of the Tool

The descriptive analysis evaluates the subjective perception of the tool using the five constructs of the adapted UTAUT model. The results show an exceptionally positive assessment in all dimensions, with a strong tendency to agree (values close to 5 on the Likert scale).

6.4.1. Detailed Descriptive Statistics

The descriptive analysis of perception was carried out with methodological rigor, following the presentation protocol used in technology acceptance studies. The mean and standard deviation were calculated to summarize the core tendency and consistency of the responses, respectively. To obtain a complete view of the dispersion of individual perceptions, the minimum and maximum values recorded in each construct were also reported. These extreme values are crucial since they delimit the total range of the student's response, allowing us to evaluate the possible existence of highly dissatisfied participants or, on the contrary, extreme enthusiasts.

Next, the detailed descriptive analysis of the constructs is presented, following the presentation protocol used in technology acceptance studies to report the central tendency (mean), consistency (standard deviation) and total dispersion (minimum and maximum) of individual perceptions. The results of this analysis are summarized in Table 9.

Table 9. Descriptive statistics of perception constructs ($N = 30$).

Construct	Minimum	Maximum	Average ($\bar{X}$)	Standard Deviation (σ)
IV. FC	3.33	5.00	4.62	0.38
I. PU	4.08	4.64	4.41	0.16
V. ITU	4.22	4.68	4.46	0.14
II. PEOU	3.67	5.00	4.58	0.32
III. SI	3.85	4.98	4.48	0.24

6.4.2. Discussion on Acceptance (UTAUT Model)

The descriptive analysis of the constructs confirms that the intervention is perceived by the students in an exceptionally positive way. Consistently high means in all predictors of the acceptance of the adapted UTAUT model demonstrate the success of the intervention design. The following apply:

- Leadership of the FC ($\bar{X} = 4.62$): This construct showed one of the highest means. Although the deviation ($\sigma = 0.38$) reflects the natural heterogeneity of a larger group ($N = 30$), the data demonstrates that the vast majority of students felt supported by the infrastructure. This result consolidates the initial finding of Section 6.2.1, validating that the effort in the design of the plugin and the server architecture removed the barriers to entry, allowing students to focus purely on the logical task.
- High PU ($\bar{X} = 4.41$) and Intention to Use ($\bar{X} = 4.46$): PU and Intention to Use showed very high averages, comparable and higher than the reference studies. This validates the main pedagogical objective: students feel that the tool improved their understanding and show a high desire to recommend and use it in the future.
- PEOU ($\bar{X} = 4.58$): The perception that the interaction was intuitive is particularly relevant given that 30% of the sample had no prior experience with Minecraft. The high

score ensures that the learning curve of the interface is low, allowing students to concentrate on the complexity of the digital logic rather than the technology itself.

- SI ($\bar{X} = 4.48$): This high average, along with the high desire for recommendation (part of UI), underscores the value of Minecraft's common environment in fostering collaborative learning and social motivation.

The results of the UTAUT model confirm that the Minecraft environment was perceived as useful, easy to use and with consolidated support, achieving its central objective of being a superior practical reinforcement tool for the teaching of digital logic.

6.5. Correlations Between Perception and Performance

The crucial stage of impact validation consisted of determining the linear relationship between objective performance (completed tasks/modules) and subjective perception (UTAUT constructs). Pearson's correlation coefficient (r) [50] was used to cross these variables, in order to test whether an experience perceived as positive enhances academic success. Pearson's correlation coefficient is a statistical measure that quantifies the strength and direction of the linear relationship between two continuous variables. In this case, the variables of the score obtained by the plugin and the means of the Likert constructs are compared.

6.5.1. Comparison Between Objective Performance and UTAUT Constructs

To determine the real impact of the environment on learning, we proceeded to calculate Pearson's Correlation Coefficient (r) between the Objective Performance metric (the Number of tasks/modules completed) and the averages of the Perception constructs (Usefulness, Ease and Intention to Use).

This analysis is performed to test the hypothesis that a positive perception of the system (a high Likert value) is directly related to a high performance in logic exercises (a high objective score). The results are summarized in Table 10. With the sample increase to $N = 30$, the statistical significance of the findings has been considerably strengthened.

Table 10. Pearson's correlation (r) between Perception and Target Performance ($N = 30$).

Construct	Coefficient r	Significance p	Implication
PU	0.81	$p < 0.001$	Very strong relationship with performance.
ITU	0.85	$p < 0.001$	The highest correlation validates the positive reinforcement cycle.
PEOU	0.58	$p < 0.01$	Significant relationship: Usability facilitates achievement.
SI	0.62	$p < 0.001$	Strong relationship: social support is linked to success.

6.5.2. Discussion of the Relationship Between Perception and Performance

The results of Pearson's Correlation coefficient (r) revealed a key finding for the project: the correlation is strong and highly significant ($r \geq 0.58$) for all core constructs that predict acceptance. All significance values (p) were less than 0.01, and in the most critical cases less than 0.001, which confirms that the observed relationships are statistically robust and not the product of chance. The following apply:

- Validation of pedagogical impact: The strongest correlation is between Performance and Intention to Use ($r = 0.85$) and PU ($r = 0.81$). This demonstrates a direct and very strong effect: the students who scored the highest were precisely those who found the

plugin more useful for learning and who expressed a greater willingness to continue using it.

- Usability as an achievement facilitator: The significant coefficient for PEOU ($r = 0.58$) confirms that a low technological barrier to entry, without interface or installation problems, not only facilitates access, but also enhances academic success in the main task of logic construction.
- Positive reinforcement cycle: These results validate that the quality of the experience (i.e., that the plugin is easy to use and perceived as useful) generates a virtuous cycle. This cycle works like this: students' high performance in completing the exercises directly drives their intention for future use. This is crucial because it guarantees the sustainability of the tool in the curriculum: if students perform correctly and enjoy the process, they will want to continue using it and consequently recommend it.

6.5.3. Correlation Between Constructs—Pearson Matrix

To validate the interdependence between the perception factors, as postulated by the UTAUT model, the Pearson correlation matrix (r) was calculated between the five constructs. This analysis verifies whether perception in one dimension (e.g., PEOU) influences the perception of another (e.g., PU).

The Pearson matrix is a table that shows the coefficient r for each pair of variables. In the context of UTAUT, this matrix is crucial to confirm the structural validity of the model.

The UTAUT model predicts that factors such as PEOU and FC should have a strong positive correlation with PU and ITU. Pearson's correlation matrix is used to verify this structure, since a high positive correlation (close to $r = 1$) between two constructs indicates that the increase in the perception of one is associated with the increase in the perception of the other.

Pearson's correlation matrix, which shows the intercorrelations between the perceptual constructs (adapted UTAUT model), can be seen in Table 11. It is observed that all correlations are positive, which validates the internal consistency of the perception model.

Table 11. Pearson Correlation Matrix (r) between constructs ($N = 30$).

Construct	PU	PEOU	SI	FC	ITU
PU	1.00	—	—	—	—
PEOU	0.49	1.00	—	—	—
SI	0.70	0.43	1.00	—	—
FC	0.58	0.76	0.51	1.00	—
ITU	0.88	0.56	0.75	0.65	1.00

The analysis confirms that all UTAUT dimensions are uniformly correlated positively and strongly ($r > 0.40$ in all cases). This result is critical, as it validates the cascading reinforcement effect in the plugin's acceptance process as follows:

- CF and FU reinforce utility: The value of r between FC and PEOU is $r = 0.76$. Such a high correlation validates that the robustness of the infrastructure and support have a direct and very strong impact on how easily the plugin interface is perceived by the learner.
- Adoption Chain (PU and UI): The results of the matrix confirm a very clear value chain in how students adopt the plugin. In essence, PU and ITU are extremely strongly correlated ($r = 0.88$), meaning that if the student feels like they are learning, they will want to use the tool in the future. This chain is solid because the high technical quality and PEOU of the plugin act as a foundation: since there are no frustrating glitches (thanks to the high FC), the student can focus on the task, quickly

perceive the usefulness of the environment, and that, in the end, is what drives him to continuous adoption.

6.6. Qualitative Analysis and Observations

Analysis of open-ended responses (Section 8 of the survey) and direct observations complemented the quantitative metrics, providing deep insight into the pedagogical value and development needs of the plugin.

Impact on Learning

The answers to the open question “What concept of logic gates became clearer to you thanks to the activity?” validate the constructivist principle of the proposal as follows:

- Theory-practice translation: Approximately 50% of the comments focused on the “Translation of truth table to physical circuit” and the “Difference between combinational and sequential logic” as the greatest help. This finding demonstrates that the virtual environment acts as an effective bridge connecting the abstraction of Boolean Algebra with the implementation of gate-level circuits, something that is often the main disconnect in theoretical courses.
- Complexity visualization: Advanced concepts such as the “Full Adder Operation”, the “use of the XOR gate” and the “effect of the clock on Flip-Flops” were frequently cited. The 3D environment allows students to visualize and debug the flow of data in real time, overcoming the difficulties of handwritten diagrams.

Qualitative observations also highlighted specific challenges. Students without prior gaming experience reported a steeper initial learning curve regarding 3D navigation compared to experienced users. Additionally, a tendency to prioritize aesthetics over logic was noted in early stages, though the restricted plot design effectively mitigated these distractions by limiting environmental interference.

6.7. Progression Analysis and Statistical Confidence

It is necessary to acknowledge a limitation regarding data collection in this pilot study: the system did not implement telemetry to record time-on-task metrics or the number of failed attempts. Therefore, the quantitative performance evaluation was based exclusively on the exercise completion rates as follows:

1. Completion funnel: Given that the course design imposes a mandatory sequential structure, Table 12 presents the results as a progression analysis. The data reveal that retention remains high in combinational logic (>93%) but suffers a notable decline when transitioning to sequential logic (80%), identifying this topic as the point of greatest difficulty in the learning curve.
2. Survey robustness: To validate the reliability of the acceptance data despite the sample size ($N = 30$), Table 13 details the 95% Confidence Intervals (CI). This interval indicates the probable range where the true population mean lies. The key finding is that the lower bound of all constructs remains consistently above the neutral value (3.0), statistically confirming that the students’ positive perception is significant and not merely a sample artifact.

Table 12. Progression funnel: Students who completed each level ($N = 30$).

Milestone/Exercise	Students Completing	Cumulative Retention (%)
1. Basic gates	30	100.0%
2. Combinational logic (Mux)	28	93.3%
3. Sequential logic (Flip-flops)	24	80.0%
4. Counters/Adders	22	73.3%

Table 13. Descriptive statistics and 95% Confidence Intervals for UTAUT constructs.

Construct	Mean (μ)	Std. Dev. (σ)	95% CI (Lower–Upper)
Performance Expectancy (PE)	4.52	0.51	[4.33–4.71]
Effort Expectancy (EE)	4.65	0.45	[4.48–4.82]
Attitude (AT)	4.70	0.40	[4.55–4.85]
Facilitating Conditions (FC)	4.30	0.65	[4.06–4.54]

7. Discussion

This section analyzes the computational implications of the proposed architecture, addressing the algorithmic complexity of the verification engine, the impact on server load under concurrent use, and the pedagogical interpretation of these technical constraints.

7.1. Algorithmic Complexity and Verification Cost

The verification engine operates as a deterministic finite automaton (DFA) that validates the circuit against a truth table. The algorithmic complexity of this validation is theoretically exponential, described by $O(2^N)$, where N is the number of input variables. The following apply:

- Per-student computational cost: For the educational exercises proposed in the “Computers I” course, the complexity is bounded by small values of N . The most complex scenario (JK Flip-Flop or Full Adder) utilizes $N = 3$ or $N = 4$ inputs (including clock and control signals), resulting in a maximum of $2^4 = 16$ state checks.
- Temporal cost (T_{total}) and human factor: The limiting factor is not the CPU processing time (which is negligible for $N \leq 4$), but the wait times defined by the game’s clock cycle. A standard cycle of approx. 40 ticks (2 s) per state has been established. This latency is a deliberate design decision: although the redstone signal could stabilize in less time, this delay is maintained to allow the student to visually observe the sequential operation of the circuit. The total time is defined as:

$$T_{total} = 2^N \cdot (T_{setup} + T_{pulse} + T_{prop}) \quad (6)$$

Consequently, a full validation takes between 10 and 32 s. While computationally slow compared to simulators like SPICE, this timeframe is necessary for the cognitive assimilation of the testing process.

7.2. System Parameterization and Sensitivity Analysis

The timing variables governing the verification engine ($T_{setup} = 40t$, $T_{prop} = 20t$, $T_{pulse} = 10t$) are not hardcoded constants but tunable system parameters exposed via the plugin’s configuration file. Their specific values for this pilot were determined through empirical testing to balance signal stability and pedagogical visualization.

A sensitivity analysis of these parameters reveals the following implications:

- Sensitivity to delay reduction: Reducing T_{prop} below 10 ticks increases the risk of “race conditions” in deep circuits. If the physical propagation of the Redstone signal takes longer than the verification window, the system reads an unstable intermediate state, leading to false negatives. Furthermore, extremely short pulses (<4 ticks) may fail to trigger certain piston mechanisms due to Minecraft’s internal update order.
- Sensitivity to delay extension: Increasing delays improves robustness against complex circuit topologies but linearly degrades the user experience. A $T_{prop} > 60$ ticks makes the validation process tedious, causing student frustration without adding significant technical reliability for standard exercises.

- **Robustness to tick-rate variability:** Crucially, the system relies on internal game-ticks rather than wall-clock time (real seconds). Therefore, if server performance degrades (e.g., dropping from 20 TPS to 10 TPS due to load), the validation logic remains consistent. The process will take longer in real-time, but the synchronization between the agent's actions and the circuit's response remains intact, decoupling verification accuracy from server lag.

7.3. Concurrency and Deployment Strategy

The developed system is conceived as a modular deployment unit (plugin + map template) designed to be hosted on a standard Spigot server. Therefore, the architectural analysis must be framed within the context of a typical academic course (e.g., “Computer Engineering I”), where the total enrollment is approximately $N \approx 200$ students.

In this operational scenario, massive concurrency ($N = 1000$ simultaneous active users) is not a design requirement for a single instance, as student activity is distributed as follows:

1. **Bounded concurrency:** Even with 200 enrolled students, laboratory sessions are typically segmented into smaller groups, and autonomous work (homework) is asynchronous. A single server instance easily handles these concurrency peaks without performance degradation.
2. **Horizontal scalability via replication:** Should the system need to accommodate a mass event or a larger cohort (e.g., >1000 users), the modular nature of the plugin allows for a simple horizontal scaling strategy. Since the state is self-contained per student, multiple independent server instances can be deployed in parallel to distribute the load, rather than engineering a single monolithic server to handle massive traffic.

7.4. Pedagogical Interpretation: Slowness as a Feature

From a pedagogical perspective, the non-instantaneous nature of the verification (T_{total}) transforms an apparent technical limitation into a valuable didactic feature. Unlike instantaneous software compilers, the sequential toggling of levers and flashing of lights during the validation process allows the student to engage in “visual debugging”. The extended delays allow the learner to physically see the system injecting case 001, observing the propagation, and then transitioning to 010. This reinforces the concept that digital verification is an exhaustive and sequential process of checking state spaces, aligning with the deep learning objectives of the engineering curriculum.

7.5. Limitations of the Pilot Study

The interpretation of the results must be contextualized within the limitations of this pilot study as follows:

- **Sample and generalizability:** The study relied on a small sample ($N = 30$) from a single institution. The use of volunteer sampling introduces a potential bias, as participants may have higher intrinsic motivation or prior interest in Minecraft (70% reported previous experience). Furthermore, the positive results may be partially attributed to a “novelty effect” and instructor influence, rather than exclusively to the didactic superiority of the tool.
- **Novelty effect:** The high engagement reported might be partially attributed to the “wow factor” of using a commercial video game in the classroom. Long-term studies are needed to determine if motivation persists once the novelty fades.
- **Prior gaming experience:** Although Minecraft is ubiquitous, students with prior gaming experience may have had a distinct advantage over non-gamers, potentially influencing the completion times or perceived ease of use.

- Instructor influence: While the verification system is automated, the presence of an enthusiastic instructor during the sessions could have introduced a positive bias in the students' subjective perception (UTAUT results).
- Diagnostic granularity (black box): The validation engine utilizes a "black box" approach, verifying the I/O truth table. While this is an intentional design decision to allow students creative freedom in their structural implementation, it presents a limitation in terms of feedback. The system cannot currently diagnose where a specific error is located within the circuit nor award partial credit for nearly correct solutions.
- Algorithmic constraints: The current implementation uses fixed sampling delays. Although this approach is robust for standard course exercises, it could theoretically fail in unusually deep circuits where signal propagation exceeds the preset wait window. However, it is important to note that the system's objective is not to support the creation of arbitrary circuits of large depth (N), but rather circuits specifically designed for learning purposes (small N), and not for the operation of complex systems.
- Assessment validity: As this was a pilot study, formal pre-test and post-test examinations were not conducted; therefore, the specific learning gains cannot be precisely quantified. The current data confirm that students successfully complete the exercises and accept the plugin, but they do not quantify direct cognitive impact. This limitation will be addressed in future iterations to develop a more robust educational system.

7.6. Future Roadmap

Based on the limitations identified in the pilot study, the technical analysis of the architecture, and the qualitative feedback from students (Section 6.6), a comprehensive development roadmap has been defined. This roadmap is structured into three main axes: functional evolution, algorithmic refinement, and methodological expansion.

7.6.1. Functional and Usability Enhancements

Responding directly to student feedback regarding usability and feature requests, the following implementations are prioritized to improve the User Experience (UX):

- Assisted debugging and diagnostic module: Currently, the "Black Box" engine only indicates if a circuit is valid or invalid. To address the 23% of students who requested error detection support, an intelligent diagnostic layer will be developed. This module will identify specific rows in the truth table where the student's circuit fails (e.g., "Error detected: Output is 0 when Inputs are 1,1"), effectively guiding the debugging process without revealing the structural solution.
- Persistence and state management: A manual save system will be implemented, as requested by 20% of users. This will allow the serialization of the world state (Redstone placement) to a database, enabling students to pause and resume complex exercises across different laboratory sessions, overcoming the current limitation of single-session volatility.
- Navigation optimization: The portal system will be redesigned to offer a "Fast Travel" menu, reducing the transit time between exercises and improving the workflow for advanced users.

7.6.2. Algorithmic and Pedagogical Evolution

To enhance the technical capabilities of the verification engine and adapt it to more complex learning scenarios, the following must apply:

- Advanced timing management: Research will be conducted on dynamic timing algorithms that adapt to circuit depth (L). Instead of using fixed sampling delays ($T_{prop} = 20t$), the system will dynamically calculate the optimal wait time based

on a preliminary graph analysis of the circuit. This aims to balance the pedagogical need for “slow” visual signal propagation with the efficiency required for deep circuit validation.

- Hybrid evaluation modes (white box): A dual architecture will be developed to allow instructors to toggle between the current “Black Box” mode (creative freedom) and a new “White Box” mode. The latter will utilize structural analysis to verify not just the output, but the topology of the circuit (e.g., ensuring a specific number of NAND gates are used), enabling restricted-constraint exercises.

7.6.3. Methodological Validation and Scalability

To scientifically validate the educational impact and ensure the system’s robustness at scale, the following must apply:

- Comparative control group (Verilog): To precisely quantify learning gains, future iterations of the course will establish a formal control group using traditional HDL tools (Verilog/Logisim). Data from both groups will be tracked to perform a rigorous quantitative comparison of learning outcomes, time-on-task, and error rates.
- Longitudinal analysis: A follow-up study will be conducted to assess long-term knowledge retention. This will measure the transferability of skills acquired in Minecraft to other domains of computer architecture in subsequent academic years.
- Scalability and multi-institutional deployment: Stress tests with massive student samples ($N > 200$) will be conducted to empirically determine the concurrency limits of the multi-agent architecture. Furthermore, collaboration with other universities will be sought to test a distributed deployment model, validating the generalizability of the tool in diverse academic contexts.

8. Conclusions

This study addressed a dual challenge: the technical complexity of synchronizing logic states within a non-deterministic game engine, and the pedagogical need to bridge the gap between abstract Boolean algebra and practical circuit design. Through the engineering of a multi-agent architecture and a specific validation plugin for Minecraft, a robust digital twin has been validated, transforming a sandbox game into a precise instrument for engineering education.

The conclusions are presented below, structured around three axes: the technical robustness of the validation engine, the effectiveness in academic performance, and the user acceptance validation (UTAUT).

8.1. Robustness of the Validation Engine and Sequential Synchronization

A critical contribution of this work is the resolution of the synchronization challenges within the Minecraft physics engine. The implementation of the software-controlled clock injection strategy proved to be decisive in eliminating false negatives caused by “race conditions” or signal propagation delays (T_{prop}) as follows:

- Deterministic validation: By overriding the in-game automatic oscillators and taking control of the clock cycle via server ticks ($T_{pulse} = 10$ ticks), the system successfully transformed an asynchronous simulation environment into a deterministic validation tool. This ensured that complex sequential circuits, such as JK flip-flops and counters, were evaluated with the same rigor as a professional simulator.
- Handling latency and jitter: The sampling logic, with conservative stabilization windows (T_{setup} and T_{hold}), effectively neutralized the variability of Redstone signal transmission. This allowed students to focus on logical design rather than debugging the game’s internal update quirks, a common frustration in unassisted Minecraft learning.

- Scalability: The architecture proved capable of handling concurrent circuit validations without server tick-rate degradation, confirming that the multi-agent delegation model is viable for larger deployments in mass-enrollment courses.

8.2. Validation of Operational Feasibility and Task Efficiency

Once the technical reliability was ensured, the objective results derived from the sample of 30 students confirmed the feasibility of the proposed architecture and its effectiveness in facilitating practical exercises as follows:

- High performance and consistency: The average number of completed tasks was 7.13 out of 8.00, which implies that 89.1% of the digital logic exercises were successfully solved by the students. Furthermore, the low standard deviation ($\sigma = 0.80$) indicates that this high performance was uniform across the group, validating the environment's ability to level the playing field regardless of individual starting points.
- Closing the theory-practice gap: The qualitative analysis revealed that approximately 50% of the students' comments highlighted the "translation of truth table to physical circuit" and the understanding of sequential logic as the greatest help. This validates that the tool fulfills its pedagogical objective: to act as a bridge to connect theoretical abstraction with tangible implementation.

8.3. User Acceptance Validation (UTAUT Model)

The application of the adapted UTAUT model confirmed the full acceptance of the tool, demonstrating that the platform has a solid foundation for its sustainability.

First, the development environment was perceived as technically solid: the FC construct had a high mean ($\bar{X} = 4.62$) and, crucially, 0% of learners reported blocking technical issues. This technical soundness removed barriers to entry, allowing students to focus purely on logic.

This success led to a high perception of value. The constructs of PU ($\bar{X} = 4.41$) and ITU ($\bar{X} = 4.46$) demonstrated an exceptionally high rating. Additionally, the PEOU score ($\bar{X} = 4.58$) is particularly significant given that 30% of the participants had no prior experience with Minecraft, proving that the designed interface is intuitive enough to be used by non-gamers.

In addition, Pearson's correlation analysis showed that this positive perception is directly and strongly linked to performance. The correlation was highly significant between Target Performance and PU ($r = 0.81$) and ITU ($r = 0.85, p < 0.001$). This creates a positive reinforcement cycle: the quality of the technical experience directly enhances academic success, which in turn promotes continued adoption.

8.4. Implications and Projection

The initiative reinforces the vision of more interactive, accessible and sustainable educational systems. The project, implemented with limited resources, aligns directly with SDG 4 (Quality Education) by providing a high-impact, reproducible tool.

However, to precisely define the scope of these conclusions, it is necessary to state what has not yet been demonstrated. While this work proves pilot feasibility and acceptance, it does not yet provide quantitative evidence of learning gains superior to traditional baselines (e.g., Verilog) due to the absence of a control group. Furthermore, the generalization of these results to other engineering domains or massive-scale contexts remains a subject for future longitudinal research.

In conclusion, this work consolidates Minecraft, assisted by an automated validation plugin, as an innovative, robust and superior educational tool for teaching the fundamental concepts of digital logic.

Author Contributions: Conceptualization, A.Á.S.; Methodology, A.Á.S. and I.A.C.; Software, D.C.G.; Validation, D.C.G. and I.A.C.; Formal analysis, D.C.G.; Investigation, D.C.G. and S.G.G.; Writing—review and editing, D.C.G. and S.G.G.; Supervision, I.A.C. and G.V.G.; Funding acquisition, G.V.G. All authors have read and agreed to the published version of the manuscript.

Funding: The APC was supported by the project Self-adaptive platform based on intelligent agents for the optimization and management of operational processes in logistic warehouses (PLAUTON), PID2023-151701OB-C21, funded by MCIN/AEI/10.13039/501100011033/FEDER, EU. The founders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

Institutional Review Board Statement: Ethical review and approval were waived for this study due to the educational and non-medical nature of the research, which involved voluntary, anonymous participation and did not collect sensitive personal data.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The data presented in this study are available within the article.

Acknowledgments: This work has been supported by the University of Salamanca through the Teaching Innovation Project ID2025/097 ("USALCRAFT: Design and implementation of a virtual platform for teaching hardware description languages and combinational circuits in engineering subjects").

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Tokatlidis, C.; Tselegkaridis, S.; Rapti, S.; Sapounidis, T.; Papakostas, D. Hands-On and Virtual Laboratories in Electronic Circuits Learning—Knowledge and Skills Acquisition. *Information* **2024**, *15*, 672. [CrossRef]
2. Sakkas, K.; Ntagka, N.E.; Spyridakis, M.; Miltiadous, A.; Glavas, E.; Tzallas, A.T.; Giannakeas, N. Multiplayer Virtual Labs for Electronic Circuit Design: A Digital Twin-Based Learning Approach. *Electronics* **2025**, *14*, 3163. [CrossRef]
3. Piszcz, A.; Rojek, I.; Náprstková, N.; Mikołajewski, D. Using VR and BCI to Improve Communication Between a Cyber-Physical System and an Operator in the Industrial Internet of Things. *Appl. Sci.* **2025**, *15*, 12805. [CrossRef]
4. Balan, G.; Neninger, P.; Ruiz Zúñiga, E.; Serea, E.; Lucache, D.-D.; Sălceanu, A. A Perspective on Software-in-the-Loop and Hardware-in-the-Loop Within Digital Twin Frameworks for Automotive Lighting Systems. *Appl. Sci.* **2025**, *15*, 8445. [CrossRef]
5. Shalash, O.; Métwalli, A.; Sallam, M.; Khatab, E. A Multimodal Polygraph Framework with Optimized Machine Learning for Robust Deception Detection. *Inventions* **2025**, *10*, 96. [CrossRef]
6. Faust, C. Using a Minecraft virtual workspace for learning digital electronics. *Am. J. Phys.* **2024**, *92*, 317–319. [CrossRef]
7. Liu, Q. Minecraft Redstone: The Gateway to Engineering. *Lect. Notes Educ. Psychol. Public Media* **2024**, *59*, 95–100. [CrossRef]
8. Ritella, G.; Marcone, R. Constructing in Minecraft in Primary School as a Boundary-Crossing Practice. *Educ. Sci.* **2024**, *14*, 85. [CrossRef]
9. Jover, V.; Sempere, S. Creating Digital Twins to Celebrate Commemorative Events in the Metaverse. *Computers* **2025**, *14*, 273. [CrossRef]
10. Wang, S.-M.; Vu, L.H. The Integration of Digital Twin and Serious Game Framework for New Normal Virtual Urban Exploration and Social Interaction. *J. Urban Manag.* **2023**, *12*, 168–181. [CrossRef]
11. Papert, S.; Harel, I. Situating Constructionism. In *Constructionism*; Ablex Publishing Corporation: Norwood, NJ, USA, 1991. Available online: <http://www.papert.org/articles/SituatingConstructionism.html> (accessed on 20 December 2025).
12. Kolb, D.A. *Experiential Learning: Experience as the Source of Learning and Development*; Prentice-Hall: Hoboken, NJ, USA, 1984. Available online: https://www.researchgate.net/publication/315793484_Experiential_Learning_Experience_as_the_source_of_Learning_and_Development_Second_Edition (accessed on 20 December 2025).
13. Hu, W.; Zhang, T.; Deng, X.; Liu, Z.; Tan, J. Digital twin: A state-of-the-art review of its enabling technologies, applications and challenges. *J. Intell. Manuf. Spec. Equip.* **2021**, *2*, 1–34. [CrossRef]
14. Bayık, N.A.; Uygun, K. The Effect of Digital Game-Based Learning Applications on Students' Problem-Solving Skills and Attitudes Toward Social Studies: The Case of Minecraft Education. *Int. J. Excell. Leadersh.* **2025**, *5*, 38–47.
15. Nebel, S.; Schneider, S.; Rey, G.D. Mining Learning and Crafting Scientific Experiments: A Literature Review on the Use of Minecraft in Education and Research. *Educ. Technol. Soc.* **2016**, *19*, 355–366.
16. Schrotter, G.; Hürzeler, C. The Digital Twin of the City of Zurich for Urban Planning. *PFG-J. Photogramm. Remote Sens. Geoinf. Sci.* **2020**, *88*, 99–112. [CrossRef]

17. Kavouras, I.; Rallis, I.; Sardis, E.; Protopapadakis, E.; Doulamis, A.; Doulamis, N. Empowering Communities Through Gamified Urban Design Solutions. *Smart Cities* **2025**, *8*, 44. [\[CrossRef\]](#)
18. Deterding, S.; Dixon, D.; Khaled, R.; Nacke, L. From game design elements to gamefulness: Defining gamification. In *15th International Academic MindTrek Conference: Proceedings of the Envisioning Future Media Environments, Tampere, Finland, 20–30 September 2011*; ACM: New York, NY, USA, 2011; pp. 9–15. [\[CrossRef\]](#)
19. Triantafyllou, S.A.; Georgiadis, C.K.; Sapounidis, T. Gamification in education and training: A literature review. *Int. Rev. Educ.* **2025**, *71*, 483–517. [\[CrossRef\]](#)
20. Huotari, K.; Hamari, J. A definition for gamification: Anchoring gamification in the service marketing literature. *Electron. Mark.* **2017**, *27*, 21–31. [\[CrossRef\]](#)
21. Gini, F.; Bassanelli, S.; Bonetti, F.; Mogavi, R.H.; Bucchiarione, A.; Marconi, A. The role and scope of gamification in education: A scientometric literature review. *Acta Psychol.* **2025**, *259*, 105418. [\[CrossRef\]](#)
22. Lai, C.-H.; Hu, P.-Y. The gaming revolution in history education: The practice and challenges of integrating game-based learning into formal education. *Information* **2025**, *16*, 490. [\[CrossRef\]](#)
23. Bunt, L.; Greeff, J.; Taylor, E. Enhancing serious game design: Expert-reviewed, stakeholder-centered framework. *JMIR Serious Games* **2024**, *12*. [\[CrossRef\]](#)
24. Lee, L.-K.; Wei, X.; Chui, K.T.; Cheung, S.K.S.; Wang, F.L.; Fung, Y.-C.; Lu, A.; Hui, Y.K.; Hao, T.; U, L.H.; et al. A systematic review of the design of serious games for innovative learning: Augmented reality, virtual reality, or mixed reality? *Electronics* **2024**, *13*, 890. [\[CrossRef\]](#)
25. Bacalja, A.; Nichols, T.P.; Robinson, B.; Bhatt, I.; Kucharczyk, S.; Zomer, C.; Nash, B.; Dupont, B.; De Cock, R.; Zaman, B.; et al. Postdigital Videogames Literacies: Thinking With, Through, and Beyond James Gee’s Learning Principles. *Postdigital Sci. Educ.* **2024**, *6*, 1103–1142. [\[CrossRef\]](#)
26. Costaguta, R.; Aciar, S.; Paderewski, P.; Gutiérrez-Vela, F. Adaptive Videogames for Education: An Initial Study. In *Human-Computer Interaction. HCI-COLLAB 2023*; Communications in Computer and Information Science; Springer: Cham, Switzerland, 2024; Volume 1877, pp. 40–49. [\[CrossRef\]](#)
27. Adžić, S.; Al-Mansour, J.; Naqvi, H.; Stambolić, S. The impact of video games on students’ educational outcomes. *Entertainment Computing* **2021**, *38*, 100412. [\[CrossRef\]](#)
28. López-Fernández, D. Comparing effectiveness of educational video games of different genres in computer science education. *Educ. Chem. Eng.* **2023**, *47*, 201–212. [\[CrossRef\]](#)
29. Angelides, M.C.; Agius, H. Video Games in Educational Settings: Developing Skills for New Media Learning. In *Handbook of Digital Games*; Wiley Online Library: Hoboken, NJ, USA, 2014; pp. 502–522. [\[CrossRef\]](#)
30. Presentación-Muñoz, A.; González-Fernández, A.; Rodal, M.; Acevedo-Borrega, J. The Use of Video Games in Language Learning: A Bibliometric Analysis. *Metrics* **2025**, *2*, 12. [\[CrossRef\]](#)
31. Slattery, E.J.; Butler, D.; O’Leary, M.; Marshall, K. Primary School Students’ Experiences Using Minecraft Education During a National Project-Based Initiative: An Irish Study. *TechTrends* **2023**, advance online publication. [\[CrossRef\]](#)
32. Triantafyllou, S.A.; Farhaoui, Y. Delving into Scientific Utilization of Minecraft: An Overview. In *Intersection of Artificial Intelligence, Data Science, and Cutting-Edge Technologies: From Concepts to Applications in Smart Environment*; Farhaoui, Y., Herawan, T., Imoize, A.L., Allaoui, A.E., Eds.; Springer Nature: Cham, Switzerland, 2025; pp. 215–228. [\[CrossRef\]](#)
33. Emiroglu, E.; Grant, C.A.; Sermet, Y.; Demir, I. Floodcraft: Game-based interactive learning environment using Minecraft for flood mitigation for K-12 education. *Int. J. Disaster Risk Reduct.* **2025**, *130*, 105799. [\[CrossRef\]](#)
34. Korte, S.-M.; Körkkö, M.; Førelund, L.R. Developing computational thinking skills in higher education through peer reflection on robotics and programming exercises with Bee-Bots, Lego Mindstorms EV3 and Minecraft Education. *Learn. Cult. Soc. Interact.* **2025**, *55*, 100947. [\[CrossRef\]](#)
35. Slattery, E.J.; Butler, D.; Marshall, K.; Barrett, M.; Hyland, N.; O’Leary, M.; McAvinue, L.P. Effectiveness of a Minecraft Education intervention for improving spatial thinking in primary school children: A mixed methods two-level cluster randomised trial. *Learn. Instr.* **2024**, *94*, 102003. [\[CrossRef\]](#)
36. Barragán-Pulido, S.; Barragán-Pulido, M.L.; Alonso-Hernández, J.B.; Castro-Sánchez, J.J.; Rabazo-Méndez, M.J. Development of Students’ Skills through Gamification and Serious Games: An Exploratory Study. *Appl. Sci.* **2023**, *13*, 5495. [\[CrossRef\]](#)
37. Höltingen, S.; Fecker, T.; Pleikies, S.; Wormsbecher, N.; Divani, S. A Case of Toy Computing Implementing Digital Logics with “Minecraft”. In *Handbook of Unconventional Computing*; Adamatzky, A., Ed.; World Scientific: Singapore, 2021; pp. 439–462. [\[CrossRef\]](#)
38. Fedorenko, E.G.; Kaidan, N.V.; Velychko, V.Y.; Soloviev, V.N. Gamification when studying logical operators on the Minecraft EDU platform. In *Proceedings of the International Workshop on Augmented Reality in Education, Kryvyi Rih, Ukraine, 11 May 2021*. Available online: <https://api.semanticscholar.org/CorpusID:235748800> (accessed on 20 December 2025).
39. Zhang, J.; Zhu, J.; Tu, W.; Wang, M.; Yang, Y.; Qian, F.; Xu, Y. The effectiveness of a digital twin learning system in assisting engineering education courses: A case of landscape architecture. *Appl. Sci.* **2024**, *14*, 6484. [\[CrossRef\]](#)

40. Weis, P.; Baš'ovanský, R.; Vereš, M. A web-based digital twin framework for interactive e-learning in engineering education. *Computers* **2025**, *14*, 435. [CrossRef]
41. Cavalcanti, A.P.; Barbosa, A.; Carvalho, R.; Freitas, F.; Tsai, Y.-S.; Gašević, D.; Mello, R.F. Automatic feedback in online learning environments: A systematic literature review. *Comput. Educ. Artif. Intell.* **2021**, *2*, 100027. [CrossRef]
42. Messer, M.; Brown, N.C.C.; Kölling, M.; Shi, M. Automated grading and feedback tools for programming education: A systematic review. *ACM Trans. Comput. Educ.* **2024**, *24*, 10. [CrossRef]
43. Škopljanač-Mačina, F.; Zakarija, I.; Blašković, B. Towards automated assessment generation in e-learning systems using combinatorial testing and formal concept analysis. *IEEE Access* **2021**, *9*, 52957–52976. [CrossRef]
44. SpigotMC. Spigot Javadocs—Server Scheduler. SpigotMC Developer Wiki. Official API Documentation for Scheduling Tasks in the Spigot Minecraft Server Framework. 2024. Available online: <https://hub.spigotmc.org/javadocs/spigot/> (accessed on 20 December 2025).
45. Department of Computer Science and Automation, University of Salamanca. *Computer Engineering I: Teaching Materials and Logic Circuit Exercises*; University of Salamanca Web Portal: Salamanca, Spain, 2024. Available online: <http://avellano.usal.es/~compi/> (accessed on 20 December 2025).
46. Akinnuwesi, B.A.; Uzoka, F.-M.E.; Fashoto, S.G.; Mbunge, E.; Odumabo, A.; Amusa, O.O.; Okpeku, M.; Owolabi, O. A modified UTAUT model for the acceptance and use of digital technology for tackling COVID-19. *Sustain. Oper. Comput.* **2022**, *3*, 118–135. [CrossRef]
47. Villarrubia, G.; De Paz, J.F.; Bajo, J.; Corchado, J.M. Ambient agents: Embedded agents for remote control and monitoring using the PANGAEA platform. *Sensors* **2014**, *14*, 13955–13979. [CrossRef]
48. Zato, C.; Villarrubia, G.; Sánchez, A.; Barri, I.; Rubión, E.; Fernández, A.; Rebate, C.; Cabo, J.A.; Álamos, T.; Sanz, J.; et al. PANGAEA—Platform for automatic construction of organizations of intelligent agents. In *Distributed Computing and Artificial Intelligence*; Communications in Computer and Information Science; Springer: Berlin/Heidelberg, Germany, 2012; pp. 229–239. [CrossRef]
49. Cajas Bravo, T.; Silva Infantes, M.; Dávila Morán, R.C. Percepción docente sobre el uso de la tecnología en la educación superior. *Conrado* **2023**, *19*, 326–335. Available online: http://scielo.sld.cu/scielo.php?script=sci_arttext&pid=S1990-86442023000100326&nrm=iso (accessed on 20 December 2025).
50. Fiallos, G. La correlación de Pearson y el proceso de regresión por el método de mínimos cuadrados. *Cienc. Lat. Rev. Científica Multidiscip.* **2021**, *5*, 2491–2509. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.